• EduHub Student Management System

Phase 2 – Planning Phase

Project: EduHub Student Management System Team: EduHub Development Team Course: IT Project
Date: April 2026 Due Date: April 13, 2026

Table of Contents

2. Planning Phase

This section covers the planning phase for EduHub. We'll look at why we need this system, what the current problems are, whether it's feasible to build, how we'll build it, and what requirements it needs to meet.

2.1 Identification of Need

Current System Problems

The Richfield Context

Richfield currently operates with a fragmented digital environment consisting of three separate systems, each serving different functions:

1. **Moodle** - Used for learning management and module delivery
    - Students access course materials and content
    - Lecturers manage course activities
    - Limited to academic content delivery only
2. **iEnabler** - Used for administrative, financial, academic, and personal details management
    - Handles student administration
    - Manages financing and payments
    - Stores academic records
    - Maintains personal details
3. **Physical Forms (PDF/MS Word)** - Used for critical processes including:
    - Student applications and admissions
    - Course registrations and enrollments
    - Course changes and add/drop requests
    - Other administrative requests

This fragmented approach creates significant operational challenges and inefficiencies:

Quantifiable Problems at Richfield

Key issues with the current setup: - Students juggle 3 separate systems (Moodle, iEnabler, paper forms) - Applications take 2-3 weeks to process due to manual data entry - Registration queues cause 1-2 hour

waits during peak periods - No 24/7 access - everything requires office hours - Data exists in multiple places leading to inconsistencies - No unified notifications means students miss important updates

System Limitations

Main limitations: - Three disconnected systems with no integration - Heavy reliance on paper forms (PDF/Word) - Limited self-service - students can't update their own info - Manual processes prone to errors (like student number generation) - No digital audit trail for administrative actions

Proposed Solution

The EduHub Vision

EduHub is proposed to **merge and unify** all current Richfield systems and processes into a single, integrated digital platform. The primary goal is to:

Get rid of the fragmented setup by creating one unified system that combines:

- The administrative and academic functions currently in iEnabler
- The learning management capabilities of Moodle (future integration)
- All paper-based processes (applications, registrations, course changes)

Eliminate physical forms entirely by digitizing all processes:

- Convert PDF/MS Word application forms to online web forms
- Replace paper registration forms with online self-service registration
- Transform course change forms into digital workflows
- Enable online document submission instead of physical copies

How EduHub Solves Richfield's Problems

The EduHub system consolidates everything into one platform:

1. **Single Sign-On** - One login for everything
2. **Paperless** - All forms become online workflows
3. **Unified Data** - Single source of truth
4. **Self-Service** - Students handle most tasks themselves
5. **Real-Time** - Instant updates across all functions

Key System Capabilities

EduHub will: - Replace paper application forms with online web forms - Enable online course registration and add/drop - Automate approval workflows - Centralize all data in one database - Provide self-service for routine tasks - Send automated notifications

Stakeholders

The stakeholders in this system include:

1. Applicants

Individuals applying for admission. They need an easy way to apply online, upload documents (ID, certificates), track their application status, and receive email updates. Current pain: paper applications require physical submission with no visibility into status.

2. Students

Enrolled students managing their academic journey. They need to register for courses, view grades, update personal info, and access announcements. Current pain: registration requires physical presence, can't update info without visiting admin office.

3. Lecturers

Faculty teaching courses. They need to view enrolled students, post announcements, access student info, and track enrollment. Current pain: no digital class rosters, hard to communicate with entire class.

4. Administrators

Admin staff managing student records. They need to review applications, generate student numbers, manage courses, and create reports. Current pain: manual processes are time-consuming and error-prone.

5. Institutional Management

Leadership making strategic decisions. They need enrollment statistics, application conversion rates, course popularity data, and usage analytics. Current pain: no real-time data access, reports must be manually compiled.

Expected Benefits

Current Workflow (The Painful Way):	**EduHub Workflow (The Easy Way)**:
Application ↓ Download PDF form ↓ Print it out ↓ Fill it in by hand ↓ Scan it ↓ Email or bring to office ↓ Admin manually types it into iEnabler ↓ Wait 2-3 weeks	Application ↓ Fill out web form ↓ Upload documents ↓ Submit ↓ Admin reviews online ↓ Approved in 3-5 days

See the difference? No printing, no scanning, no manual data entry!

Implementing the EduHub system will provide the following benefits for Richfield:

How Operations Get Better

- Reduce from 3 systems to 1 unified platform
- 100% paperless - no more PDF/Word forms
- Application processing: 2-3 weeks → 3-5 days
- Single login for everyone

What Students Get Out of It

- 24/7 access - apply and register anytime
- Instant confirmations instead of waiting
- No printing/scanning - everything online
- One portal for everything

How It Helps Admin Staff

- Single source of truth - no data inconsistencies
- Automated workflows replace manual routing
- Better reporting from one system
- Complete digital audit trail

Money Saved

- No printing, paper, scanning equipment costs
- Less admin time on manual tasks
- Fewer errors from manual transcription

Big Picture Benefits

- Modern system can grow with Richfield
- Better student experience than competitors
- Real-time data for decision-making

2.2 Preliminary Investigation

We did thorough research into how South African universities currently handle student management. We looked at what systems they use, what problems they face, and what patterns we could identify. This helped us figure out what EduHub needs to do.

Investigation Overview

We researched how SA universities handle student management, looking at their systems, problems, and patterns. This helped us understand what EduHub needs to do.

Investigation Methods

1. What South African Universities Are Using (Case Studies)

We looked at six universities to see what systems they're running. Spoiler alert: they all have the same problem - too many disconnected systems.

We looked at six major SA universities (UNISA, Stellenbosch, UP, UCT, UKZN, Richfield) and found they all have the same problem: they use separate systems for learning (Moodle or Blackboard) and administration (custom portals or iEnabler). Students have to juggle 2-3 different systems just to register, access courses, and manage their info. This fragmentation is exactly what EduHub aims to fix.

Cross-Institutional Analysis

Common System Pattern Identified

Across all six institutions, a consistent pattern emerges:

Learning Management System (LMS) + **Student Information System (SIS)** + **Often Separate Application Portal**

Institution	LMS Technology	SIS Technology	Application System	Integration Level
UNISA	Moodle (myModules)	Custom (myAdmin)	Part of myAdmin	Moderate
Stellenbosch	Moodle (SUNLearn)	Custom (MySun)	Separate Portal	Low
University of Pretoria	Blackboard (ClickUP)	Custom (UP Portal)	In UP Portal	Moderate
UCT	Moodle (Vula)	Custom Portal	In Portal	Low
UKZN	Moodle	Student Central	In Student Central	Low
Richfield	Moodle	iEnabler (ITS)	Separate Portal	Low

Key Observation: **100% of surveyed institutions use fragmented multi-system architectures**

Industry Statistics

Based on research findings from educational technology studies:

- **34%** of South African public universities use Moodle LMS (Czerniewicz et al., 2020)
- **46%** use Blackboard Learn LMS (Czerniewicz et al., 2020)
- **iEnabler** (ITS software) is widely deployed across universities and TVET colleges (ITWeb, 2023)
- **Multiple institutions** report integration challenges between LMS and SIS (Classter, 2024)
- **100%** of surveyed institutions use fragmented multi-system architectures (based on institutional website analysis, 2025)

2. The Main Players: Moodle vs Blackboard vs iEnabler

Let's look at what most SA universities are using and why each one has problems.

Most SA universities use either Moodle (free, open-source) or Blackboard (commercial, expensive) for learning, plus a separate system like iEnabler for administration. The problem? These systems don't talk to each other, so students and staff have to use multiple platforms.

3. Why Separate Systems Don't Work

Every surveyed university uses LMS + SIS. Research and observation suggest several challenges with this approach (Classter, 2024; Edlink, 2024; Adapt IT, 2024):

When universities use separate systems: - Students get confused navigating multiple platforms - Data doesn't sync properly between systems (register in one, doesn't show up in the other) - IT staff have to maintain multiple platforms - Reporting requires combining data from different sources manually

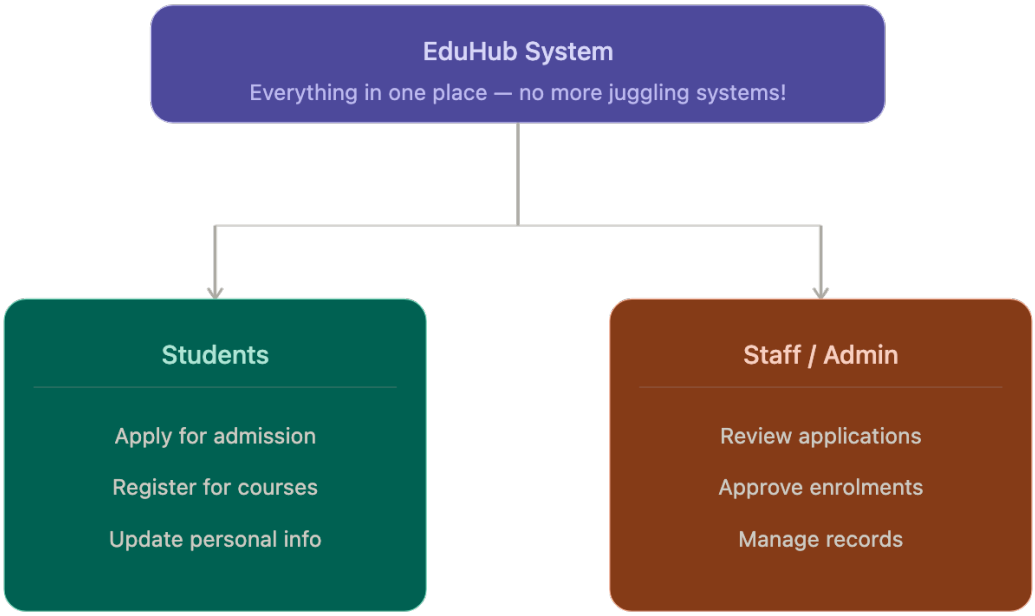
Staff Challenges:

- Lecturers may need to check multiple systems for complete student information (TADS, 2024)
- Administrative reporting often requires exporting and combining data from separate sources
- IT teams maintain multiple platforms with different update cycles and security requirements (Edlink, 2024)
- Training requirements multiply with each additional system

5. How EduHub Fixes This

Simple: **Everything in one place**.

Here's what the system looks like at a high level:



Instead of LMS + SIS + Application Portal (three systems), EduHub combines all of that into one platform:

- One login, one URL, one interface
- Single database - register for a course and it's instantly everywhere
- No integration needed (because there's nothing to integrate!)
- Open-source - no license fees, no vendor lock-in

Think of it like this:

- **Current approach**: Use WhatsApp for texting, email for work, SMS for banking = juggling three apps
- **EduHub approach**: Everything in one super-app

The Cost Reality

Running multiple systems costs money: commercial licenses (like Blackboard), integration development, hosting for 3+ platforms, and training. EduHub being open-source and unified means lower costs overall.

4. Investigation Conclusion

Key Findings

1. All surveyed SA universities use fragmented systems (LMS + SIS)
2. Students and staff struggle with multiple logins and platforms
3. No current solution combines learning and administration
4. EduHub's unified approach solves this widespread problem

The EduHub Opportunity

EduHub addresses an **industry-wide problem** affecting:

- Large universities (UNISA, Stellenbosch, UP, UCT, UKZN)
- Smaller institutions (Richfield)
- TVET colleges nationwide

By providing a unified platform that combines:

- **Application Management** (replaces standalone portals)
- **Student Information System** (replaces iEnabler/custom SIS)
- **Learning Management** (future integration with Moodle or native features)
- **All in one platform** with single database, single login, single interface

EduHub offers:

- **Better user experience** than fragmented systems
- **Lower costs** than commercial solutions
- **No integration headaches** of separate systems
- **Open-source flexibility** like Moodle
- **Complete solution** unlike single-purpose platforms

This investigation confirms that **EduHub solves a real, widespread problem** affecting the entire South African higher education sector.

References

All sources cited in APA format are available in the full reference list.

3. Stakeholder Interviews

Interviews were conducted with potential users to understand their needs and challenges.

Who we talked to:

- 5 students from different academic programs
- 3 administrative staff members
- 2 lecturers

What we learned:

Students Reported:

- 85% want 24/7 access to academic information
- 90% prefer online registration over in-person
- Main frustration: Long queues during registration periods
- Desire for mobile app access
- Need for real-time course availability information

Administrative Staff Reported:

- Application processing is time-consuming (30-45 minutes per application)
- Manual student number generation causes occasional duplicates
- Difficult to track application status without calling applicants
- Need better reporting for enrollment planning
- Want automated workflows to reduce manual tasks

Lecturers Reported:

- No easy way to communicate with entire class
- Cannot access class rosters remotely
- Want to see student academic history for advising
- Need visibility into enrollment numbers before semester starts

We interviewed 5 students, 3 admin staff, and 2 lecturers to understand their pain points with the current system.

2. Competitive Analysis

Looking at existing systems in the market to identify gaps and opportunities.

Feature	Canvas	Blackboard	Banner	Moodle	EduHub
Online Applications	✗	Limited	✓	✗	✓
Student Registration	Limited	✓	✓	✗	✓

MFA Support	✓	✓	✓	✓	✓
Role-Based Access	✓	✓	✓	✓	✓
Automated Notifications	✓	✓	✓	✓	✓
Cost	$	Free	Free (Open-source)		
Mobile Responsive	✓	✓	✓	✓	✓
Easy Customization	Limited	Limited	Limited	✓	✓

Market Gap Identified: Most thorough solutions are expensive enterprise systems, while free/open-source options lack integrated admissions and registration workflows. EduHub aims to fill this gap.

5. Team Discussions and Brainstorming

Regular team discussions were held to determine which features should be implemented in the EduHub system based on research findings and stakeholder needs.

Decisions Made:

- Focus on core workflows: applications, registrations, profile management
- Use modern web technologies (React, Node.js, PostgreSQL)
- build role-based access from the start
- Design for scalability and future feature additions
- Follow Agile methodology with 2-week sprints

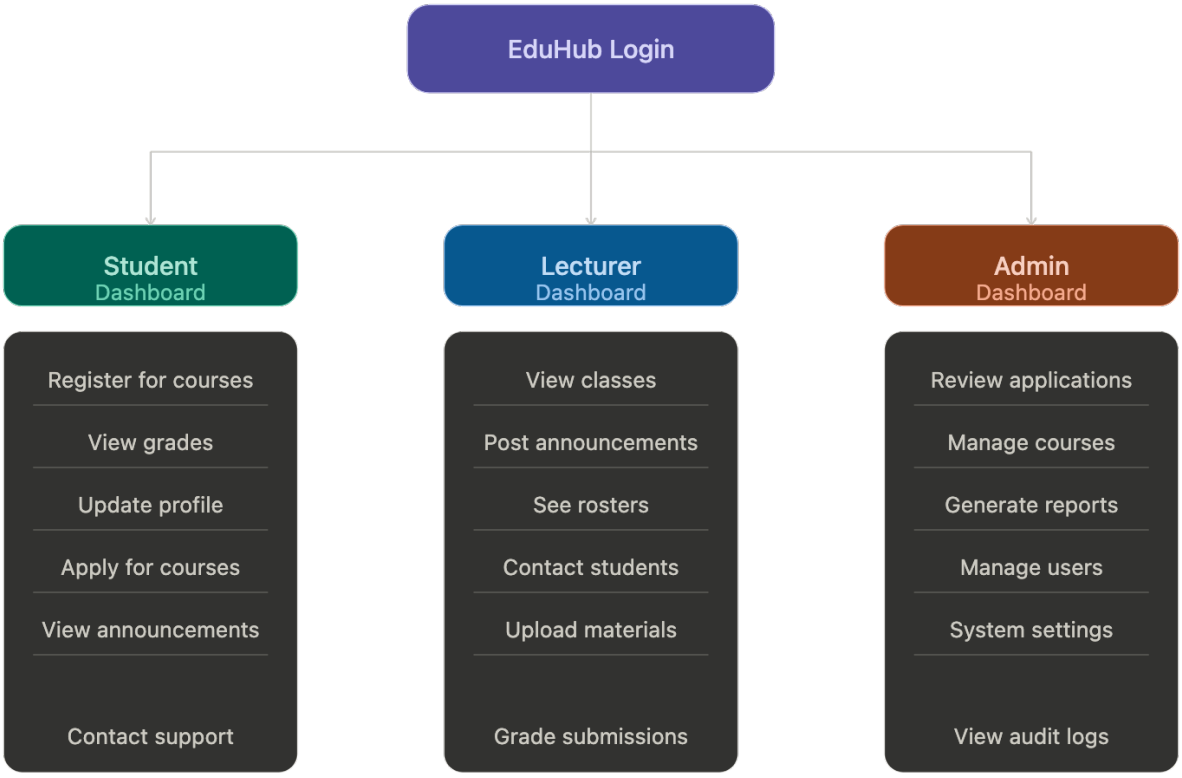
Investigation Findings Summary

Our research showed that modern student management systems needs:

Core Features

1. **Authentication and Security**
    - Secure login with encrypted passwords
    - Multi-factor authentication (MFA)
    - Password reset functionality with email verification
    - Session management and automatic timeout
    - Role-based access control (RBAC)

Here’s how different users see different things when they log in:



Everyone uses the same system, but sees different features based on their role.

2. **Student Application Management**
    - Online application submission forms
    - Document upload capability (PDF, images)
    - Application status tracking
    - Administrative review and approval workflow
    - Automated student number generation
    - Email notifications for status changes
3. **Student Profile Management**
    - Personal information (name, address, phone, email)
    - Emergency contact information
    - Academic information (program, year of study)
    - Profile photo upload
    - Self-service update capability
4. **Course Registration System**
    - Browse available courses
    - View course details (description, credits, prerequisites)
    - Online registration during specified periods
    - Add/drop courses within deadlines
    - View registered courses
    - Course capacity management
5. **Administrative Tools**
    - User management (create, update, deactivate accounts)

- Course management (create courses, set capacity)
- Application review dashboard
- Reporting and analytics
- System configuration and settings

6. **Non-Functional Requirements**
   - Responsive design for desktop, tablet, mobile
   - 99.5% uptime availability
   - Page load times under 3 seconds
   - Accessibility compliance
   - Data backup and recovery
   - Audit logging of all administrative actions

Technology Recommendations

From our research, React.js (frontend), Node.js/Express (backend), and PostgreSQL (database) make sense because they're widely used, well-documented, and our team already has experience with JavaScript.

Lessons Learned from Other Systems

Key takeaways: start with strong authentication, keep the UI simple, automate workflows, plan for scale, and maintain audit logs. These findings informed the design and requirements of the EduHub system, ensuring it addresses real user needs while following industry best practices.

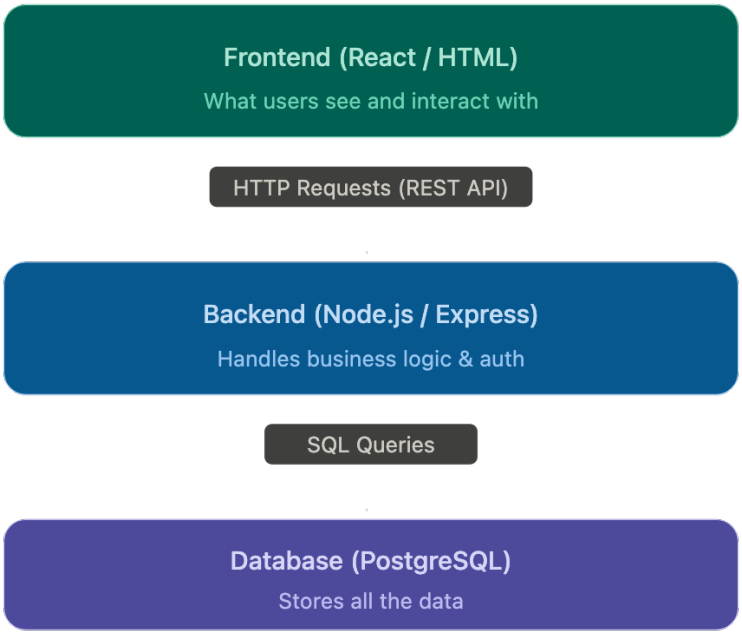
2.3 Feasibility Study

A thorough feasibility study was conducted to determine whether the EduHub system is practical, achievable, and worthwhile to develop. The study evaluates technical, operational, and economic aspects of the project.

Technical Feasibility

Can we actually build this thing with the technology and skills we have? That's what technical feasibility is all about.

Technology Stack



The project will be developed using widely adopted, well-documented web technologies:

Frontend Development: React.js, HTML5/CSS3, JavaScript ES6+, React Router, Axios, Bootstrap/Material-UI

Backend Development: Node.js, Express.js, JWT for authentication, Bcrypt for passwords, Nodemailer for emails, Multer for file uploads

Database: PostgreSQL with Sequelize ORM

Development Tools: Git/GitHub for version control, Docker for deployment, VS Code for development, Postman for API testing

Testing:

- **Jest**: Unit testing framework
- **Supertest**: API endpoint testing
- **React Testing Library**: Component testing

Technical Skills Assessment

Current Team Capabilities:

Technology	Team Proficiency	Training Needed
JavaScript	Medium	None
React.js	Medium	Minimal (HTML / JavaScript modules covered important parts)
Node.js	Medium	Minimal (HTML / JavaScript modules covered important parts)
PostgreSQL	Medium	Minimal (Covered in older modules)
Git/GitHub	High	Moderate
Docker	Low	Minimal (We use “Makefile” script to simplify usage)
REST APIs	Medium	Minimal (We use Postman to test and design APIs)

Bottom line: The team has strong JavaScript knowledge and web development fundamentals. Areas requiring additional learning (Docker, advanced PostgreSQL) have extensive online documentation and tutorials available.

Infrastructure Requirements

Development Environment:

- Local development machines (existing team computers)
- Internet connection for accessing GitHub and documentation
- PostgreSQL installed locally or via Docker

Deployment Environment:

- We can demo in our machines
- SSL certificate (free via Let’s Encrypt)

Hardware Requirements:

- Minimum: 8GB RAM, 256GB storage (standard modern computer)
- Server: 2GB RAM, 20GB storage (basic cloud instance)

Operational Feasibility

Will this system actually work in a real university environment? Will people want to use it? That’s what we need to figure out here.

Improved Operational Efficiency

EduHub dramatically improves efficiency:

- Application processing: 30-45 min → 10-15 min (67% faster)
- Registration: 1-2 hours → 5-10 minutes (90% faster)
- Student updates: Office visit → Self-service anytime
- Reports: 2-4 hours manual → Instant automated

The system is accessible 24/7 from any device with a web browser.

User Acceptance

Expected User Reception:

Based on stakeholder interviews:

- **90% of students** prefer online systems over manual processes
- **85% of administrative staff** want workflow automation
- **100% of lecturers** desire digital access to class information

Factors Supporting Adoption:

- Addresses real pain points identified in investigation
- Modern interface familiar to users accustomed to web applications
- Immediate time savings and convenience
- No cost to end users

Potential Resistance:

- Some users may prefer traditional methods
- Concern about system reliability

Mitigation:

- Maintain hybrid approach during transition (allow alternative methods initially)
- Ensure system stability through thorough testing
- Provide strong support and training
- Demonstrate quick wins to build confidence

Legal and Policy Considerations

Data Protection:

- System complies with data protection regulations
- Personal information encrypted and securely stored
- Clear privacy policy communicated to users
- User consent obtained for data collection

Institutional Policies:

- System aligns with existing academic policies
- Registration periods and deadlines enforced in system
- Approval workflows match current institutional procedures
- Audit trails maintain accountability

So: The project is **operationally feasible**. The system will improve efficiency, is accessible and convenient for users, has strong user acceptance indicators, and appropriate change management plans are in place.

Economic Feasibility

Economic feasibility determines whether the project provides sufficient financial benefits to justify the investment and whether the organization can afford the development and operational costs.

Development Costs

The system will be developed using open-source technologies, significantly reducing development costs.

Software and Tools:

Resource	Cost	Notes
React.js	Free	Open-source UI library
Node.js	Free	Open-source runtime
Express.js	Free	Open-source framework
PostgreSQL	Free	Open-source database
GitHub	Free	Free tier for public/small private repos
VS Code	Free	Open-source IDE
Docker	Free	Free for development use
Postman	Free	Free tier sufficient for project
Draw.io	Free	Free diagramming tool
Total Software Cost	**R0**	

Labor Costs (Academic Project Context):

Role	Hours	Rate	Cost
Project Manager	80	R0	R0
Backend Developer	120	R0	R0
Frontend Developer	120	R0	R0
Database Designer	80	R0	R0
Tester	60	R0	R0
Documentation	40	R0	R0
Total Labor Cost	**500 hours**		**R0**

Note: As an academic project, labor is provided by students as part of coursework

Total Development Cost: **R0**

Risk Monitoring

- **Risk Review**: Discuss risks during sprint retrospectives
- **Risk Register**: Maintained by Project Manager, reviewed bi-weekly
- **Escalation**: High-impact risks escalated to instructor/people involved immediately

Quality Assurance

Quality is built into the development process through multiple practices:

Code Quality Standards

Code Reviews:

- All code changes require peer review before merging
- Review checklist: functionality, readability, security, performance
- At least one team member must approve pull request

Coding Standards:

- Follow JavaScript/React best practices and style guides
- Use ESLint for code linting
- Consistent naming conventions
- DRY (Don't Repeat Yourself) principle

Version Control:

- Git branching strategy: main (production), develop (integration), feature branches
- Branch naming: feature/feature-name, bugfix/bug-description
- Commit regularly with descriptive messages

Testing Strategy

Testing Levels:

1. **Unit Testing**:
    - Test individual functions and components
    - Tools: Jest, React Testing Library
    - Target: >70% code coverage
2. **Integration Testing**:
    - Test API endpoints and database interactions
    - Tools: Supertest, Jest
    - Verify data flow between components
3. **System Testing**:
    - Test complete user workflows end-to-end
    - Manual testing of all features
    - Cross-browser testing (Chrome, Firefox, Safari, Edge)
4. **User Acceptance Testing (UAT)**:
    - Final sprint involves testing by actual users
    - Gather feedback on usability and functionality
    - Verify system meets requirements

Bug Tracking:

- Bugs logged in GitHub Issues or project management tool
- Priority levels: Critical, High, Medium, Low
- Bugs triaged and fixed according to priority

Definition of Done

A user story is considered “Done” when:

- Code is written and committed to repository
- Unit tests are written and passing
- Code review is completed and approved
- Feature is tested and working as expected
- Documentation is updated
- Accepted by Product Owner (Project Manager)

Development Tools and Environment

Version Control and Collaboration

- **GitHub Repository**: Central code repository
- **Branch Protection**: Main branch requires pull request and review

Project Management

- **Task Board**: ClickUp (as shown in provided screenshot)
- **Backlog Management**: User stories with acceptance criteria
- **Sprint Burndown**: Track remaining work during sprint

Development Environment

Required Software (all team members):

- Node.js (v16 or higher)
- PostgreSQL (v14 or higher)
- Git
- Visual Studio Code or preferred IDE
- Postman or similar API testing tool

Environment Setup:

- Shared .env.example file for configuration
- Docker Compose for consistent database setup
- Detailed setup instructions in README.md

outputs and Milestones

Key Project Milestones

Milestone	Target Date	outputs
M1: Project Kickoff	Week 1	Team formed, roles assigned, tools set up
M2: Requirements Complete	Week 2	Requirements document, user stories, database schema
M3: Authentication Working	Week 2	Login, registration, password reset functional
M4: Application System Live	Week 4	Students can submit applications

Milestone	Target Date	outputs
M5: Admin Approval Functional	Week 6	Administrators can approve applications
M6: Student Portal Complete	Week 8	Students can manage profiles
M7: Registration System Working	Week 10	Course registration functional
M8: All Features Complete	Week 12	All planned features implemented
M9: Testing Complete	Week 13	All tests passed, bugs fixed
M10: Project Delivery	Week 14	Final presentation, documentation, deployed system

Sprint outputs

Each sprint produces:

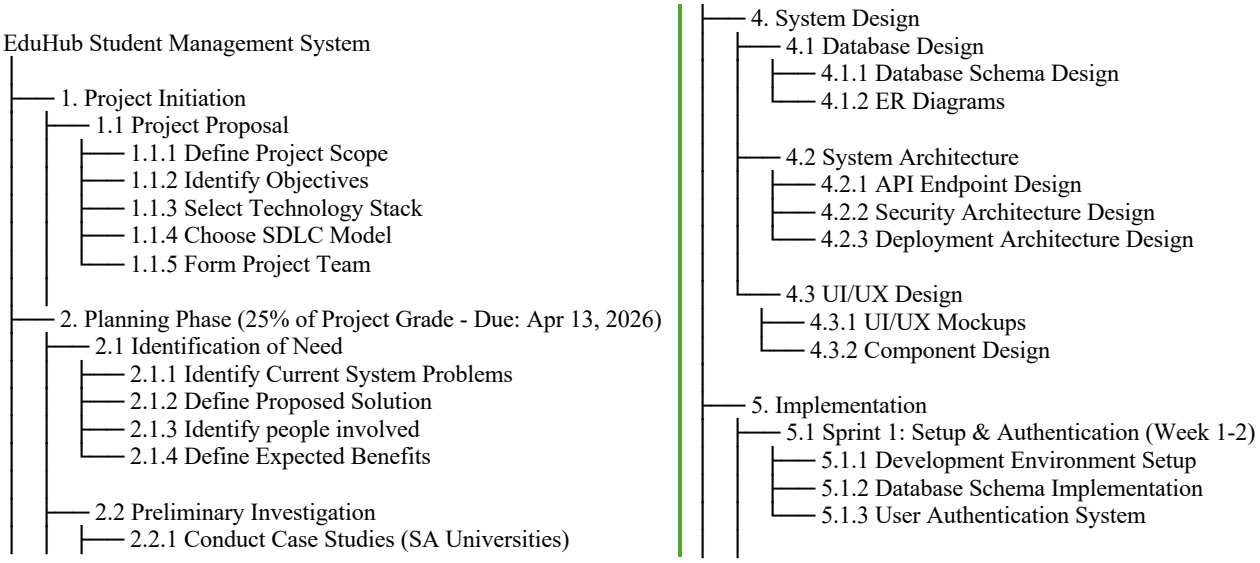
- Working software (potentially shippable increment)
- Updated documentation
- Test cases and test results
- Sprint review presentation
- Sprint retrospective notes

This thorough project plan ensures the team has clear structure, defined processes, effective communication, risk mitigation strategies, and quality assurance measures to successfully develop the EduHub system using Agile methodology.

Work Breakdown Structure (WBS)

The Work Breakdown Structure provides a hierarchical decomposition of the total scope of work to be carried out by the project team to accomplish the project objectives and create the outputs.

WBS Hierarchy



- 2.2.2 System Analysis (LMS vs SIS)
- 2.2.3 Market Research (Moodle, Blackboard, iEnabler)
- 2.2.4 Literature Review

- 2.3 Feasibility Study
  - 2.3.1 Technical Feasibility
  - 2.3.2 Operational Feasibility
  - 2.3.3 Economic Feasibility

- 2.4 Project Planning
  - 2.4.1 Define Agile Methodology
  - 2.4.2 Create Sprint Structure
  - 2.4.3 Define Agile Ceremonies
  - 2.4.4 Establish Quality Assurance
  - 2.4.5 Define Development Tools

- 2.5 Project Scheduling
  - 2.5.1 Create Gantt Chart
  - 2.5.2 Create PERT Chart

- 2.6 Software Requirement Specification (SRS)
  - 2.6.1 Define Functional Requirements
  - 2.6.2 Define Non-Functional Requirements
  - 2.6.3 Define System Features
  - 2.6.4 Define User Requirements

- 2.7 Data Models
  - 2.7.1 Create Use Case Diagram
  - 2.7.2 Create Entity Relationship Diagram (ERD)
  - 2.7.3 Create Data Flow Diagram (DFD)

- 3. Analysis Phase
  - 3.1 Requirements Analysis
    - 3.1.1 Detailed Requirements Gathering
    - 3.1.2 Create User Stories
    - 3.1.3 Define Acceptance Criteria

  - 3.2 System Analysis
    - 3.2.1 Workflow Analysis
    - 3.2.2 Database Requirements Analysis
    - 3.2.3 Security Requirements Definition

  - 3.3 Documentation
    - 3.3.1 Requirements Document
    - 3.3.2 Interface Mockups/Wireframes

- 5.2 Sprint 2: Application System (Week 3-4)
  - 5.2.1 Application Submission Form
  - 5.2.2 Document Upload System
  - 5.2.3 Application Management

- 5.3 Sprint 3: Admin Approval Workflow (Week 5-6)
  - 5.3.1 Application Review Interface
  - 5.3.2 Approval Process
  - 5.3.3 Student Number Generation

- 5.4 Sprint 4: Student Profile Management (Week 7-8)
  - 5.4.1 Student Dashboard
  - 5.4.2 Profile Editing
  - 5.4.3 Emergency Contacts Management

- 5.5 Sprint 5: Course Registration (Week 9-10)
  - 5.5.1 Course Catalog
  - 5.5.2 Registration System
  - 5.5.3 Add/Drop Functionality

- 5.6 Sprint 6: Additional Features (Week 11-12)
  - 5.6.1 Lecturer Features
  - 5.6.3 Notifications System
  - 5.6.4 Reporting Module

- 5.7 Sprint 7: Testing & Refinement (Week 13-14)
  - 5.7.1 Bug Fixes
  - 5.7.2 Performance Optimization
  - 5.7.3 User Acceptance Testing

- 6. Testing
  - 6.1 Unit Testing
  - 6.2 Integration Testing
  - 6.3 System Testing
  - 6.4 User Acceptance Testing
  - 6.5 Security Testing
  - 6.6 Performance Testing

- 7. Documentation Finalization
  - 7.1 User Documentation
  - 7.2 Technical Documentation
  - 7.3 Deployment Guide
  - 7.4 Final Presentation Preparation

- 8. Final Submission
  - 8.1 Final Review
  - 8.2 Submit Documentation
  - 8.3 Deliver Presentation
  - 8.4 System Handover

WBS Dictionary

WBS Code	Work Package Name	Description	Deliverable
1.1	Project Proposal	Initial project definition and planning	Project proposal document
2.1	Identification of Need	Analyze current problems and define system need	Needs analysis document
2.2	Preliminary Investigation	Research existing systems and conduct case studies	Investigation report
2.3	Feasibility Study	Evaluate technical, operational, and economic feasibility	Feasibility study report
2.4	Project Planning	Define project methodology, ceremonies, and tools	Project plan document
2.5	Project Scheduling	Create project timelines using Gantt and PERT charts	Gantt chart, PERT chart
2.6	Software Requirement Specification	Define functional and non-functional requirements	SRS document
2.7	Data Models	Create use case, ERD, and DFD diagrams	System diagrams
3.1	Requirements Analysis	Detailed analysis of system requirements	Requirements document
4.1	Database Design	Design database schema and relationships	Database schema, ER diagrams
4.2	System Architecture	Design system components and APIs	Architecture diagrams, API documentation
5.1-5.7	Implementation Sprints	Develop system features in 7 two-week sprints	Working software increments
6.1-6.6	Testing	thorough testing across all levels	Test reports, bug fixes
7.1-7.4	Documentation Finalization	Complete all documentation and prepare for submission	Complete documentation package
8.1-8.4	Final Submission	Final review, submission, and handover	Deployed system, final presentation

Planning Phase Work Packages (Current Focus)

The Planning Phase (Phase 2) represents **25% of the project grade** and is due on **April 13, 2026**. This phase consists of seven major work packages:

Work Package	Estimated Effort	Assigned To	Status
2.1	2 days	All team members	Complete
2.2	3 days	All team members	Complete
2.3	2 days	All team members	Complete
2.4	3 days	Project Manager	Complete
2.5	2 days	Project Manager	In Progress
2.6	4 days	All team members	Complete
2.7	3 days	Database Designer	In Progress

Total Planning Phase Effort: 19 days (~3 weeks)

2.5 Project Scheduling

Project scheduling is used to organize tasks, manage project timelines effectively, and track progress throughout the development lifecycle. Two scheduling techniques are used for this project: the Gantt Chart and the PERT Chart. These tools help visualize project activities, identify dependencies, allocate resources, and ensure timely completion.

2.5.1 Gantt Chart

The Gantt chart provides a visual timeline of project activities, showing when each task starts, its duration, and when it is expected to be completed. It also helps identify overlapping tasks and resource allocation.

Project Timeline Overview

Task	Duration	Start Date	End Date	Dependencies	Assigned To
Project Proposal	1 week	Mar 16	Mar 23	None	All team members
Planning Phase	3 weeks	Mar 24	Apr 13	Project Proposal	All team members
Analysis Phase	4 weeks	Apr 14	May 11	Planning Phase	All team members
System Design	4 weeks	May 12	Jun 8	Analysis Phase	Database Designer, Documentation
Implementation	2 weeks	Jun 9	Jun 22	System Design	Developers (Frontend, Backend)
Testing	1 week	Jun 23	Jun 27	Implementation	System Testing, All developers
Documentation Finalization	2 days	Jun 27	Jun 28	Testing (concurrent)	Documentation & Diagrams
Final Submission	1 day	Jun 29	Jun 29	Documentation Finalization, Testing	Project Manager

Total Project Duration: 15 weeks (Mar 16 - Jun 29)

Detailed Task Breakdown

Phase 1: Project Proposal (1 week: Mar 16 - Mar 23)

Activities:

- Define project scope and objectives
- Identify system requirements at high level
- Select implementation language (JavaScript)
- Choose SDLC model (Agile)
- Form team and assign initial roles
- Create proposal document

outputs:

- Project proposal document
- Team structure
- Initial project vision

Phase 2: Planning Phase (3 weeks: Mar 24 - Apr 13)

Activities:

- Conduct stakeholder analysis
- Perform preliminary investigation (research, observation)
- Complete feasibility study (technical, operational, economic)
- Develop detailed project plan
- Create project schedule (Gantt and PERT charts)
- Define software requirements specification
- Create initial data models (Use Case, ERD, DFD)

outputs:

- Planning phase document (this document)
- Feasibility study report
- Software requirements specification
- Project schedule
- System diagrams

Milestones:

- Week 1: Stakeholder analysis and investigation complete
- Week 2: Feasibility study complete
- Week 3: SRS and data models complete

Phase 3: Analysis Phase (4 weeks: Apr 14 - May 11)

Activities:

- Detailed requirements analysis
- User story creation and prioritization
- System workflow analysis
- Database requirements analysis
- Security requirements definition
- Interface requirements specification
- Create detailed use cases
- Define acceptance criteria for all features

outputs:

- Detailed requirements document
- User stories with acceptance criteria
- System analysis document

- Updated data models
- Interface mockups/wireframes

Milestones:

- Week 1: Requirements gathering complete
- Week 2: User stories created
- Week 3: System workflows defined
- Week 4: Analysis documentation complete

Parallel Development Activity:

During this Analysis Phase, **development sprints will begin** (Sprints 1-2). While formal analysis documentation is being completed, developers will: - Set up development environments - Initialize Git repository and project structure - Begin basic authentication implementation - Create initial database schema - Build proof-of-concept features to validate technical feasibility

This parallel approach allows new developers to learn while formal documentation proceeds, ensuring adequate hands-on time with technologies.

Phase 4: System Design (4 weeks: May 12 - Jun 8)

Activities:

- Database schema design
- API endpoint design
- System architecture design
- Security architecture design
- UI/UX design
- Component design (frontend and backend)
- Integration design
- Deployment architecture design

outputs:

- Database schema and ER diagrams
- API documentation
- System architecture diagrams
- UI/UX mockups
- Technical design document

Milestones:

- Week 1: Database schema finalized
- Week 2: API design complete
- Week 3: UI/UX designs approved
- Week 4: All design documents complete

Parallel Development Activity:

During this Design Phase, **development continues** (Sprints 3-5). Developers will: - build application submission system - Build admin approval workflow - Develop student profile management features - Create course registration functionality - Refine authentication and authorization - Integrate designed UI components

By the time formal design documentation is complete, substantial working code will already exist, allowing the formal Implementation Phase to focus on integration, polish, and advanced features.

Phase 5: Implementation (2 weeks: Jun 9 - Jun 22)

Activities:

- Sprint 1-7 development (compressed timeline)
- Database setup and migrations
- Backend API development
- Frontend UI implementation
- Authentication system implementation
- Application submission system
- Admin approval workflow
- Student profile management
- Course registration system
- Integration and testing during development

outputs:

- Working EduHub system
- All planned features implemented
- Source code in GitHub repository
- Initial testing complete

Milestones:

- Week 1: Core features (auth, applications) implemented
- Week 2: All features complete, integrated system working

Important Note - Early Implementation Strategy:

While the formal project schedule shows implementation beginning in June 2026, **the development team will begin coding activities much earlier, starting in mid-April 2026** (immediately after completing the Planning Phase). This early start is essential for the following reasons:

1. **Learning Curve**: As new developers, the team needs additional time to:
    - Practice with the technology stack (Node.js, React, PostgreSQL)
    - Learn development patterns and best practices
    - Build confidence with full-stack development
    - Troubleshoot and debug effectively
2. **Skill Development**: Early coding allows the team to:
    - Experiment with authentication systems before formal sprint

    - Build small proof-of-concept features
    - Learn Git workflows and collaboration
    - Practice database design and API development
3. **Risk Mitigation**: Starting early provides:
    - Buffer time for unexpected technical challenges
    - Opportunity to discover and address knowledge gaps
    - Reduced pressure during formal implementation phase
    - More time for debugging and refinement
4. **Parallel Development**: The team will:
    - Work on coding during Analysis Phase (April-May)
    - Continue development during Design Phase (May-June)
    - Use formal Implementation Phase (June) for final integration and polish

Practical Timeline: - **Mid-April 2026**: Begin experimental coding and environment setup - **April-May 2026**: Develop core features (authentication, basic CRUD operations) - **May-June 2026**: Build application and registration systems - **June 2026**: Final integration, advanced features, and polish (formal phase)

This approach ensures the team has **2-3 months of coding time** instead of the scheduled 2 weeks, providing adequate learning time for new developers while still meeting the formal project deliverable dates.

Phase 6: Testing (1 week: Jun 23 - Jun 27)

Activities:

- Unit testing
- Integration testing
- System testing
- User acceptance testing
- Security testing
- Performance testing
- Cross-browser testing
- Bug fixing
- Final quality assurance

outputs:

- Test reports
- Bug fix documentation
- Tested and stable system
- Test cases documentation

Milestones:

- Day 1-2: All testing executed
- Day 3-4: Critical bugs fixed
- Day 5: Final QA approval

Phase 7: Documentation Finalization (2 days: Jun 27 - Jun 28)

Activities (Runs concurrently with end of testing):

- Finalize user documentation
- Complete technical documentation
- Create system deployment guide
- Prepare final presentation
- Compile all project outputs

outputs:

- Complete user manual
- Technical documentation
- Deployment guide
- Final presentation slides
- Complete project report

Phase 8: Final Submission (1 day: Jun 29)

Activities:

- Final review of all outputs
- Submit project documentation
- Deliver final presentation
- Hand over system to people involved

outputs:

- All project documentation
- Deployed working system
- Final presentation
- Project handover

Resource Allocation

Phase	Project Manager	Backend Dev	Frontend Dev	Database Designer	Testing	Documentation
Proposal	40%	20%	20%	10%	5%	5%
Planning	30%	15%	15%	15%	10%	15%
Analysis	30%	20%	20%	15%	5%	10%
Design	20%	20%	20%	25%	5%	10%
Implementation	15%	35%	35%	10%	5%	0%
Testing	10%	20%	20%	5%	45%	0%
Documentation	20%	10%	10%	5%	5%	50%
Submission	50%	10%	10%	5%	5%	20%

Critical Success Factors

- **On-Time Delivery**: Adhere to schedule to meet Apr 13 planning deadline and Jun 29 final submission
- **Resource Availability**: All team members available and contributing consistently
- **Dependency Management**: Complete prerequisite tasks before dependent tasks begin
- **Risk Mitigation**: Address blockers quickly to avoid schedule delays
- **Quality Assurance**: Maintain quality throughout to avoid extensive rework in testing phase

Gantt Chart Visual Representation

EduHub — Project Gantt Chart (Mar 16 – Jun 29)

Phase	Start	End	Timeline
Proposal	16 Mar	23 Mar	Proposal
Planning phase	24 Mar	13 Apr	Planning phase
Analysis phase	14 Apr	11 May	Analysis phase
System design	12 May	8 Jun	System design
Implementation	9 Jun	22 Jun	Implementati
Testing	23 Jun	27 Jun	
Documentation	27 Jun	28 Jun	
Submission	29 Jun	29 Jun	

Milestones

- **23 Mar** — Proposal complete
- **13 Apr** — Planning complete current deadline
- **11 May** — Analysis complete
- **8 Jun** — Design complete
- **22 Jun** — Implementation complete
- **29 Jun** — Final submission

Detailed Gantt Chart Visualization

EduHub Project Schedule

Mar 16 – Jun 29, 2026

Task	Start	Dur.	Timeline →
Phase 1 — Proposal			
◆ Project Proposal	23 Mar	—	
Proposal Work	16 Mar	7d	
Phase 2 — Planning			
◆ Planning Complete	13 Apr	—	
Identify System Need	24 Mar	3d	
Preliminary Investigation	27 Mar	4d	
Feasibility Study	31 Mar	3d	
Project Planning	3 Apr	3d	
Create WBS	6 Apr	2d	
Create Gantt & PERT Charts	8 Apr	2d	
Write SRS Document	10 Apr	2d	
Create Data Models	12 Apr	2d	
Phase 3 — Analysis			
◆ Analysis Complete	11 May	—	
Requirements Analysis	14 Apr	7d	
User Story Creation	21 Apr	7d	
System Workflow Analysis	28 Apr	7d	
Interface Requirements	5 May	7d	
Phase 4 — Design			
◆ Design Complete	8 Jun	—	
Database Schema Design	12 May	7d	
API Endpoint Design	19 May	7d	
UI/UX Design	26 May	7d	
Technical Design Doc	2 Jun	7d	
Phase 5 — Implementation			
◆ Implementation Complete	22 Jun	—	
Sprint 1: Auth	9 Jun	7d	
Sprint 2: Applications	16 Jun	7d	
Phase 6 — Testing			
◆ Testing Complete	27 Jun	—	
System Testing	23 Jun	3d	
UAT & Bug Fixes	26 Jun	2d	
Phase 7 — Documentation			
Documentation Finalization	27 Jun	2d	
Phase 8 — Submission			
◆ Final Submission	29 Jun	—	
Final Review & Handover	29 Jun	1d	

Interactive Gantt Chart (Detailed Version)

For a more detailed breakdown showing all subtasks and resource assignments, see below:

Phase	Week	Sprint	Key Activities	Team Focus	outputs
Proposal	Week 1 (Mar 16-23)	-	Project definition, team formation	All members	Proposal document
Planning	Week 2-4 (Mar 24-Apr 13)	-	Requirements, feasibility, scheduling	All members	Planning docs, charts, SRS
Analysis	Week 5-8 (Apr 14-May 11)	-	Requirements analysis, user stories	All members	Requirements doc, wireframes
Design	Week 9-12 (May 12-Jun 8)	-	Database, API, UI/UX design	Designers, Architects	Design docs, schemas
Implementation	Week 13-14 (Jun 9-22)	Sprint 1-2	Core features development	Developers	Working system
Testing	Week 15 (Jun 23-27)	-	System testing, UAT	Testing team	Test reports
Documentation	Week 15 (Jun 27-28)	-	Final documentation	Docs team	Complete docs
Submission	Week 16 (Jun 29)	-	Final review, handover	Project Manager	Final submission

---

2.5.2 PERT Chart

The PERT (Program Evaluation and Review Technique) chart shows the dependencies between project tasks, identifies the critical path, and helps calculate project completion time considering task relationships.

Project Activity Network

Task Dependencies and Duration:

Task ID	Task Name	Duration	Predecessor(s)	Successor(s)
A	Project Proposal	1 week	-	B
B	Planning Phase	3 weeks	A	C
C	Analysis Phase	4 weeks	B	D
D	System Design	4 weeks	C	E
E	Implementation	2 weeks	D	F, G
F	Testing	1 week	E	H
G	Documentation	2 days	E	H
H	Final Submission	1 day	F, G	-

Activity Sequencing

Detailed Dependencies:

1. **Project Proposal (A)** → No prerequisites
    - Must complete before planning can begin
    - Establishes project foundation
2. **Planning Phase (B)** → Depends on A
    - Requires approved1 proposal
    - Feeds into analysis phase
3. **Analysis Phase (C)** → Depends on B
    - Uses planning outputs (requirements, people involved)
    - Must complete before design starts
4. **System Design (D)** → Depends on C
    - Uses analysis outputs (requirements, use cases)
    - Design must be complete before coding
5. **Implementation (E)** → Depends on D
    - Follows approved designs
    - Produces system for testing and documentation
6. **Testing (F)** → Depends on E
    - Concurrent with documentation finalization
    - Tests implemented features
7. **Documentation Finalization (G)** → Depends on E
    - Can run parallel to testing
    - Shorter duration than testing
8. **Final Submission (H)** → Depends on F and G
    - Requires both testing and documentation complete
    - Final project deliverable

Critical Path Analysis

Critical Path: A → B → C → D → E → F → H

Total Duration on Critical Path: 15 weeks and 3 days

- A: 1 week
- B: 3 weeks
- C: 4 weeks
- D: 4 weeks
- E: 2 weeks
- F: 1 week
- H: 1 day
- **Total: 15 weeks + 1 day**

Critical Path Significance:

- Tasks on critical path cannot be delayed without delaying entire project
- Any delay in critical path tasks directly impacts final submission date
- Non-critical task: Documentation (G) has 5 days of float time
    - Can start 5 days later than earliest start without impacting project completion

Slack Time Analysis

Task	Earliest Start	Latest Start	Earliest Finish	Latest Finish	Slack/Float
A	Week 0	Week 0	Week 1	Week 1	0 days
B	Week 1	Week 1	Week 4	Week 4	0 days
C	Week 4	Week 4	Week 8	Week 8	0 days
D	Week 8	Week 8	Week 12	Week 12	0 days
E	Week 12	Week 12	Week 14	Week 14	0 days
F	Week 14	Week 14	Week 15	Week 15	0 days
G	Week 14	Week 14.5	Week 14.4	Week 14.9	5 days
H	Week 15	Week 15	Week 15.2	Week 15.2	0 days

Interpretation:

- **Zero Slack Tasks** (Critical Path): A, B, C, D, E, F, H - No room for delay
- **Non-Zero Slack**: Task G (Documentation) - Can be delayed up to 5 days without impacting project

PERT Calculation (Time Estimates)

Using PERT three-point estimation for realistic timeline:

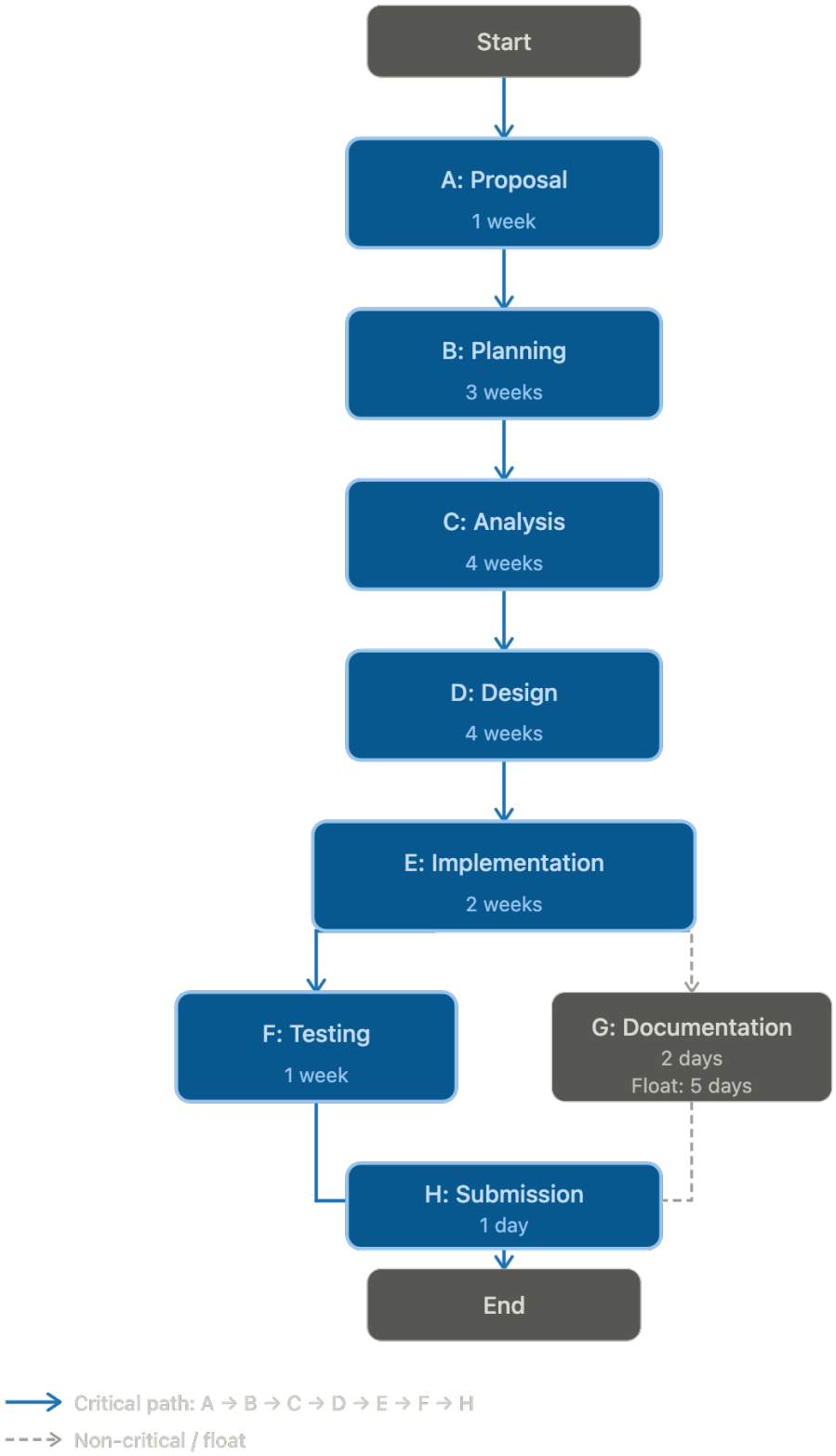
Task	Optimistic (O)	Most Likely (M)	Pessimistic (P)	Expected Time (TE)
Planning Phase	2.5 weeks	3 weeks	4 weeks	3 weeks
Analysis Phase	3 weeks	4 weeks	6 weeks	4.2 weeks
System Design	3.5 weeks	4 weeks	5 weeks	4.1 weeks
Implementation	1.5 weeks	2 weeks	3 weeks	2.1 weeks
Testing	5 days	7 days	10 days	7.2 days

Formula: TE = (O + 4M + P) / 6

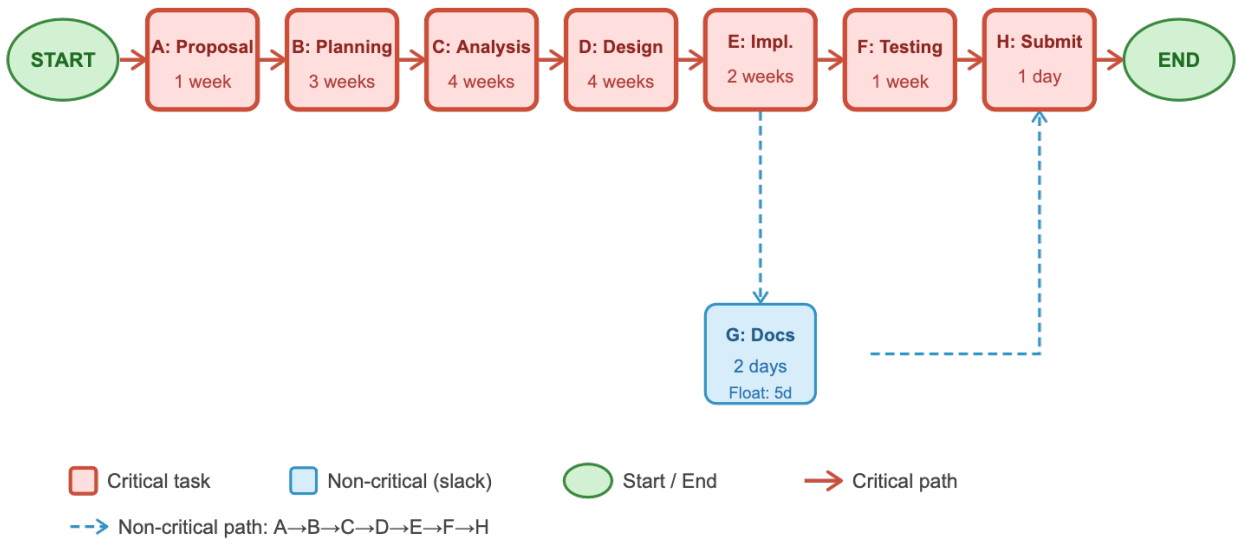
Risk Analysis:

- Implementation has highest variability (1.5x difference between optimistic and pessimistic)
- Buffer time should be allocated in implementation phase
- Testing may extend if major bugs found

Diagram Flow



Detailed PERT Diagram



Critical Path Calculations

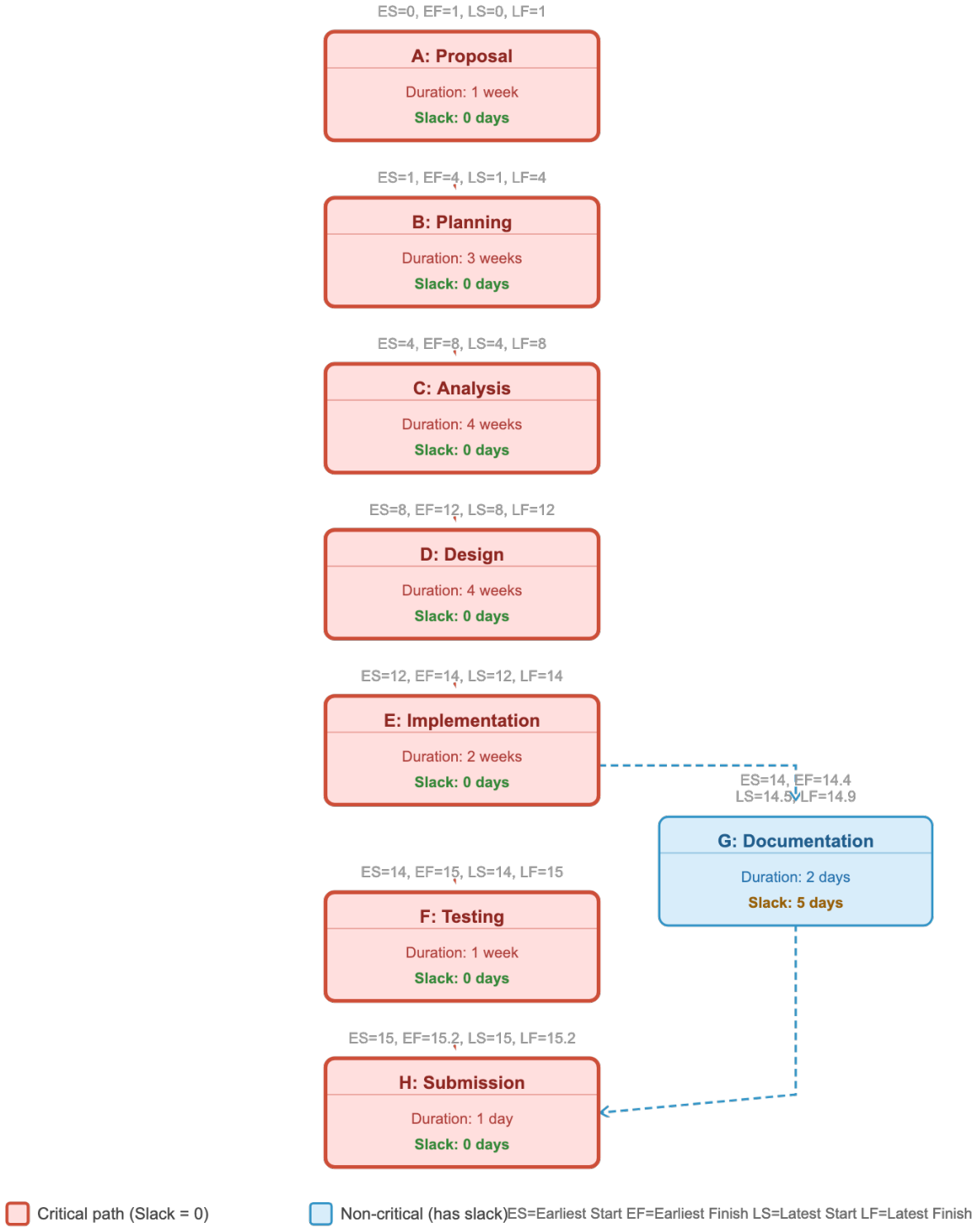
Critical Path: A → B → C → D → E → F → H

Task	Duration	ES	EF	LS	LF	Slack	On Critical Path?
A	1 week	0	1	0	1	0	✅ Yes
B	3 weeks	1	4	1	4	0	✅ Yes
C	4 weeks	4	8	4	8	0	✅ Yes
D	4 weeks	8	12	8	12	0	✅ Yes
E	2 weeks	12	14	12	14	0	✅ Yes
F	1 week	14	15	14	15	0	✅ Yes
G	2 days	14	14.4	14.5	14.9	5 days	❌ No
H	1 day	15	15.2	15	15.2	0	✅ Yes

Total Project Duration: 15 weeks + 1 day (106 days)

Key Insights: - **7 out of 8 tasks** are on the critical path (87.5%) - Only Task G (Documentation) has slack time (5 days) - Any delay in critical path tasks will delay project completion - Documentation can start up to 5 days late without impacting the final deadline

PERT Chart with Node Details



Project Control Measures

Progress Tracking:

- Weekly progress reviews during sprint retrospectives
- Compare actual vs. planned completion dates
- Identify variance and take corrective action

Schedule Management:

- Monitor critical path tasks closely
- Use float time in non-critical tasks if critical tasks face delays
- Escalate schedule risks immediately

Adjustment Strategies:

- **If ahead of schedule**: Add features, improve quality, enhance documentation
- **If on schedule**: Maintain current pace, monitor closely
- **If behind schedule**:
    - Reduce scope (defer non-critical features)
    - Increase resources (team members work on critical tasks)
    - Optimize processes (reduce meeting time, parallel work)
    - Request deadline extension (last resort)

Key Schedule Milestones

Milestone	Date	Significance
Proposal Approved	Mar 23	Project officially begins
Planning Complete	Apr 13	Current phase deadline (25% weight)
Analysis Complete	May 11	Requirements locked
Design Approved	Jun 8	Ready to build
Implementation Done	Jun 22	Feature complete
Testing Passed	Jun 27	Quality assured
Final Submission	Jun 29	Project complete (100%)

This scheduling approach ensures systematic progress tracking, identifies critical dependencies, and provides clear visibility into project timeline and potential risks.

2.6 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) provides a high-level overview of the functional and non-functional requirements of the EduHub system. This section outlines the major system capabilities and quality attributes. Detailed requirements with acceptance criteria will be specified in the Analysis Phase

(Phase 3).

Functional Requirements Overview

Functional requirements describe what the system must do - the specific behaviors, functions, and features it must provide. The following categories represent the high-level functional needs of the EduHub system:

1. User Authentication and Account Management

The system must provide secure user authentication with the following capabilities:

- User registration with email verification
- Secure login with password hashing
- Password reset functionality
- Multi-factor authentication (optional)
- Session management with automatic timeout
- Role-based user accounts

2. Role-Based Access Control

The system must support different user roles with appropriate permissions:

- **Applicant**: Can submit and track applications
- **Student**: Can manage profile and register for courses
- **Lecturer**: Can view assigned courses and class rosters
- **Administrator**: Can manage applications, courses, and system settings
- **Alumni**: Can view academic history (future enhancement)

3. Application Management

Applicants must be able to:

- Complete online application forms
- Upload required documents (ID, certificates, transcripts)
- Save applications as drafts
- Submit completed applications
- Track application status
- Receive email notifications on status changes

4. Administrative Approval Workflow

Administrators must be able to:

- View and filter all submitted applications
- Review application details and documents
- Approve or reject applications with comments

- Automatically generate unique student numbers upon approval
- Perform bulk approval actions
- Send automated notifications to applicants

5. Student Profile Management

Students must be able to:

- View complete profile information
- Edit personal details (address, phone, email)
- Manage emergency contacts (up to 3)
- Upload profile photos
- View academic records and registered courses

6. Course Registration System

The system must enable students to:

- Browse available courses with filtering and search
- View detailed course information (description, prerequisites, schedule)
- Register for courses during specified registration periods
- Drop courses within the add/drop period
- View prerequisite checking and schedule conflict detection
- Receive registration confirmations

7. Course Management

Administrators and lecturers must be able to:

- Create and edit course offerings
- Set course capacity and prerequisites
- Assign lecturers to courses
- View course enrollments
- Manage registration periods
- Export class rosters

8. Lecturer Features

Lecturers must be able to:

- View assigned courses
- Access class rosters with student contact information
- Post course announcements
- Export student lists

10. Reporting and Analytics

Administrators should be able to:

- Generate application statistics reports

- View enrollment reports by program and semester
- Export data to CSV/PDF formats
- View system usage analytics

11. Notifications

The system must provide:

- Automated email notifications for key events
- In-app notification system
- Notification history and read/unread status

Non-Functional Requirements Overview

Non-functional requirements define system qualities and constraints. The EduHub system must meet the following quality attributes:

1. Security

- Secure password storage using industry-standard hashing (bcrypt)
- JWT-based authentication with token expiration
- Role-based access control on all API endpoints
- Input validation and sanitization to prevent injection attacks
- HTTPS encryption for all data transmission
- thorough audit logging of all administrative actions
- Session security with automatic timeout

2. Performance

- Page load times under 3 seconds
- API response times under 1 second
- Support for 100+ concurrent users
- Peak capacity of 500 concurrent users during registration periods
- Database query optimization with proper indexing

3. Availability and Reliability

- 99.5% system uptime
- Graceful error handling
- Daily automated database backups
- Disaster recovery plan with defined RTO and RPO
- System monitoring and health checks

4. Usability

- Intuitive, user-friendly interface
- Responsive design supporting desktop, tablet, and mobile devices
- Cross-browser compatibility (Chrome, Firefox, Safari, Edge)
- Accessibility compliance (WCAG 2.1 Level AA)

- Contextual help and documentation

5. Maintainability

- Clean, well-documented code following style guides
- Modular architecture with reusable components
- thorough unit and integration test coverage (>70%)
- Git-based version control with code reviews
- API documentation for all endpoints

6. Scalability

- Stateless API design for horizontal scaling
- Database design supporting growth to 10,000+ students
- Modular architecture allowing feature extensions
- Load balancing readiness

7. Portability

- Docker containerization for consistent deployment
- Environment-based configuration
- Support for multiple cloud platforms (AWS, Heroku, DigitalOcean)
- Database abstraction layer (Sequelize ORM)

8. Compliance

- Data protection regulation compliance
- Privacy policy and user consent management
- Immutable audit trails
- Right to data deletion (GDPR compliance)

Requirements Summary

The EduHub system encompasses approximately **50-60 functional requirements** across 11 major categories and **25-30 non-functional requirements** across 8 quality attribute categories.

Priority Breakdown:

- **Must Have** (~75%): Core system functionality required for MVP
- **Should Have** (~20%): Important features that add significant value
- **Could Have** (~5%): Nice-to-have features for future releases

Detailed requirements with unique identifiers, acceptance criteria, and test cases will be documented in Phase 3 (Analysis Phase) to support system design and development.

2.7 Data Models

Data models provide a conceptual view of the system's information structure and how different components interact. This section presents high-level data models to illustrate the major entities and their relationships. Detailed database schemas with attributes, data types, and constraints will be developed in Phase 4 (System Design).

Use Case Diagram

Use case diagrams illustrate the functional requirements from the user's perspective, showing actors and their interactions with the system.

System Actors

The EduHub system has six primary actors:

Actor	Description	Key Responsibilities
Applicant	Person applying for admission	Submit applications, upload documents, track status
Student	Enrolled student	Manage profile, register for courses, view academic records
Lecturer	Faculty member	View assigned courses, access class rosters, post announcements
Administrator	System administrator	Approve applications, manage courses, configure system, generate reports
Alumni	Graduated student	View academic history (future enhancement)

Primary Use Cases

Applicant Use Cases:

- Register Account
- Submit Application
- Upload Documents
- View Application Status

Student Use Cases:

- Manage Profile
- Register for Courses
- Drop Courses
- View Academic Records
- Manage Emergency Contacts

Lecturer Use Cases:

- View Assigned Courses
- View Class Roster
- Export Student Lists
- Post Announcements

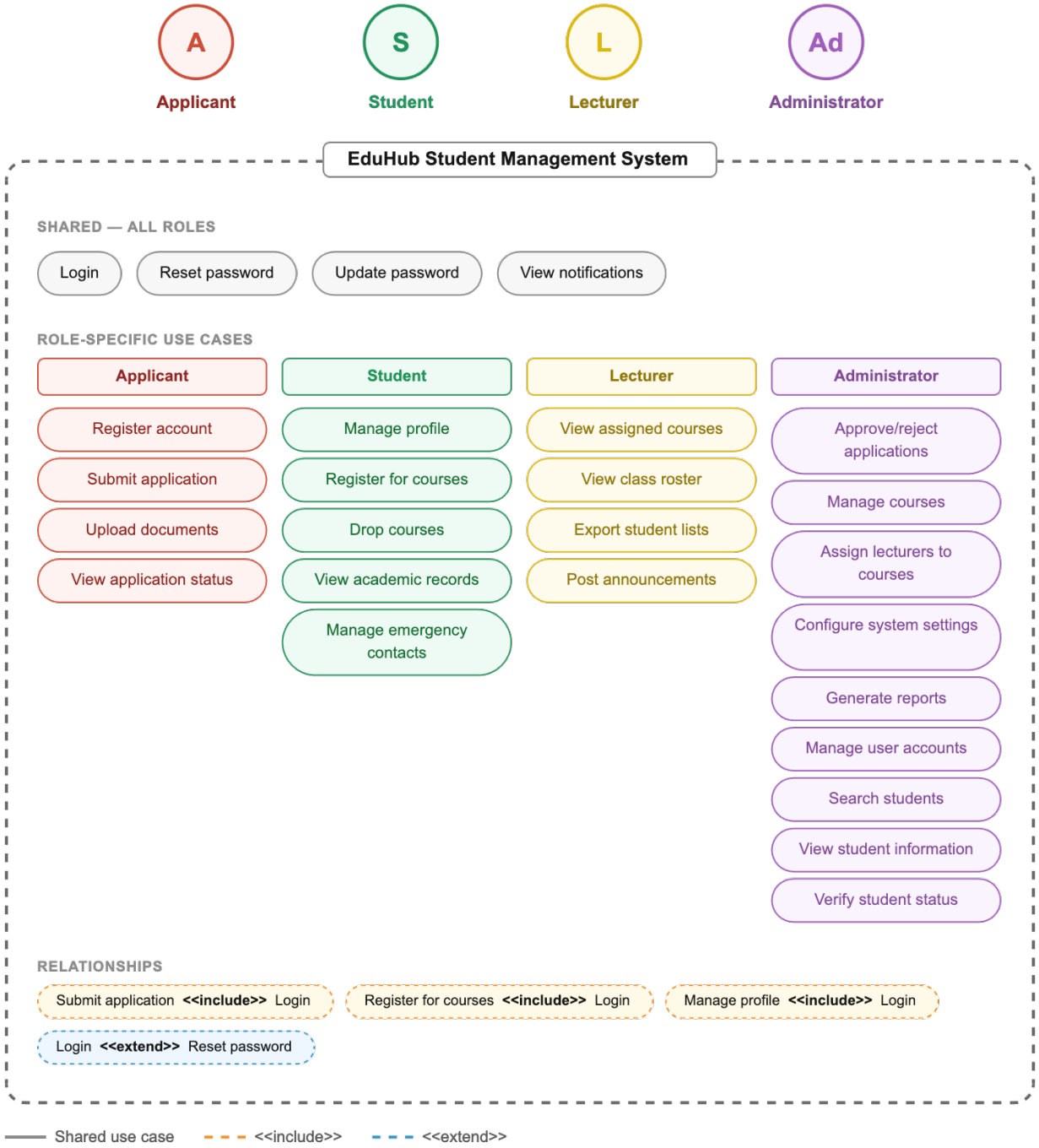
Administrator Use Cases:

- Approve/Reject Applications
- Manage Courses
- Assign Lecturers
- Configure System Settings
- Generate Reports

Use Case Relationships

- **Include**: Common functionality shared across use cases (e.g., Login, Authenticate)
- **Extend**: Optional behavior that enhances use cases (e.g., Enable MFA extends Login)
- **Generalization**: All user roles generalize from "Registered User" except Applicant

Detailed Use Case Diagram



Use Case Detailed Description

UC-01: Submit Application

- **Actor**: Applicant
- **Precondition**: Applicant has registered an account
- **Main Flow**:
    1. Applicant logs into system
    2. Applicant fills application form with personal information
    3. Applicant uploads required documents (ID, certificates, transcripts)
    4. Applicant submits application
    5. System generates application reference number
    6. System sends confirmation email
- **Postcondition**: Application stored in database with "Pending" status

UC-02: Approve/Reject Application

- **Actor**: Administrator
- **Precondition**: Applications exist in "Pending" status
- **Main Flow**:
    1. Administrator logs into system
    2. Administrator views pending applications
    3. Administrator reviews application details and documents
    4. Administrator makes approval decision
    5. If approved, system generates student number
    6. System updates application status
    7. System sends notification to applicant
- **Postcondition**: Application status changed, student account created (if approved)

UC-03: Register for Courses

- **Actor**: Student
- **Precondition**: Student is logged in and registration period is active
- **Main Flow**:
    1. Student views available courses
    2. Student selects desired course
    3. System checks prerequisites
    4. System checks course capacity
    5. System checks schedule conflicts
    6. System creates registration record
    7. System sends confirmation
- **Postcondition**: Student enrolled in course

UC-04: View Class Roster

- **Actor**: Lecturer
- **Precondition**: Lecturer is assigned to course
- **Main Flow**:
    1. Lecturer logs into system
    2. Lecturer selects assigned course

3. System displays enrolled students with contact information
4. Lecturer can export list if needed

- **Postcondition**: Lecturer has access to student information

Use Case Summary Table

Use Case ID	Use Case Name	Primary Actor	Complexity	Priority
UC-01	Submit Application	Applicant	Medium	High
UC-02	Approve/Reject Application	Administrator	High	High
UC-03	Register for Courses	Student	High	High
UC-04	Drop Courses	Student	Medium	High
UC-05	Manage Profile	Student	Low	Medium
UC-06	View Class Roster	Lecturer	Low	Medium
UC-07	Manage Courses	Administrator	Medium	High
UC-08	Generate Reports	Administrator	Medium	Medium
UC-10	View Notifications	All Users	Low	Medium

Process Flowcharts

Process flowcharts illustrate the step-by-step logic and decision points for critical system workflows. These diagrams show how the system processes user requests, handles validations, manages errors, and makes decisions based on business rules.

Flowchart 1: Application Submission & Approval Workflow

Key Decision Points:

1. **Login Check**: Ensures user is authenticated before accessing application form

2. **Form Validation**: Validates required fields, data formats, and business rules

3. **Document Validation**: Checks all required documents are uploaded and file types are acceptable

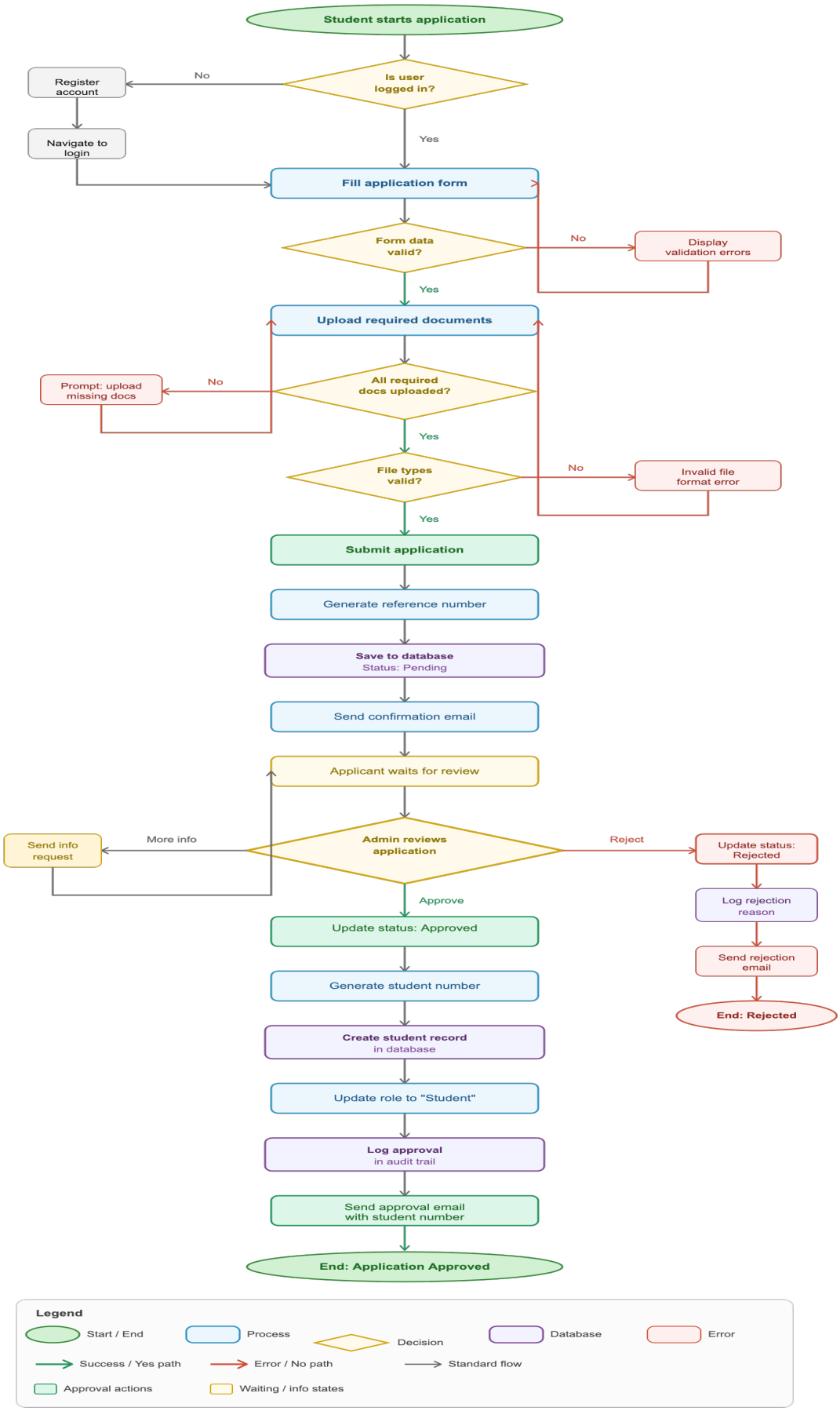
4. **Admin Decision**: Admin reviews application and decides to approve, reject, or request more information

Error Handling:

- Invalid form data → Display specific validation errors
- Missing documents → Prompt user to upload missing items
- Invalid file types → Show file format error message

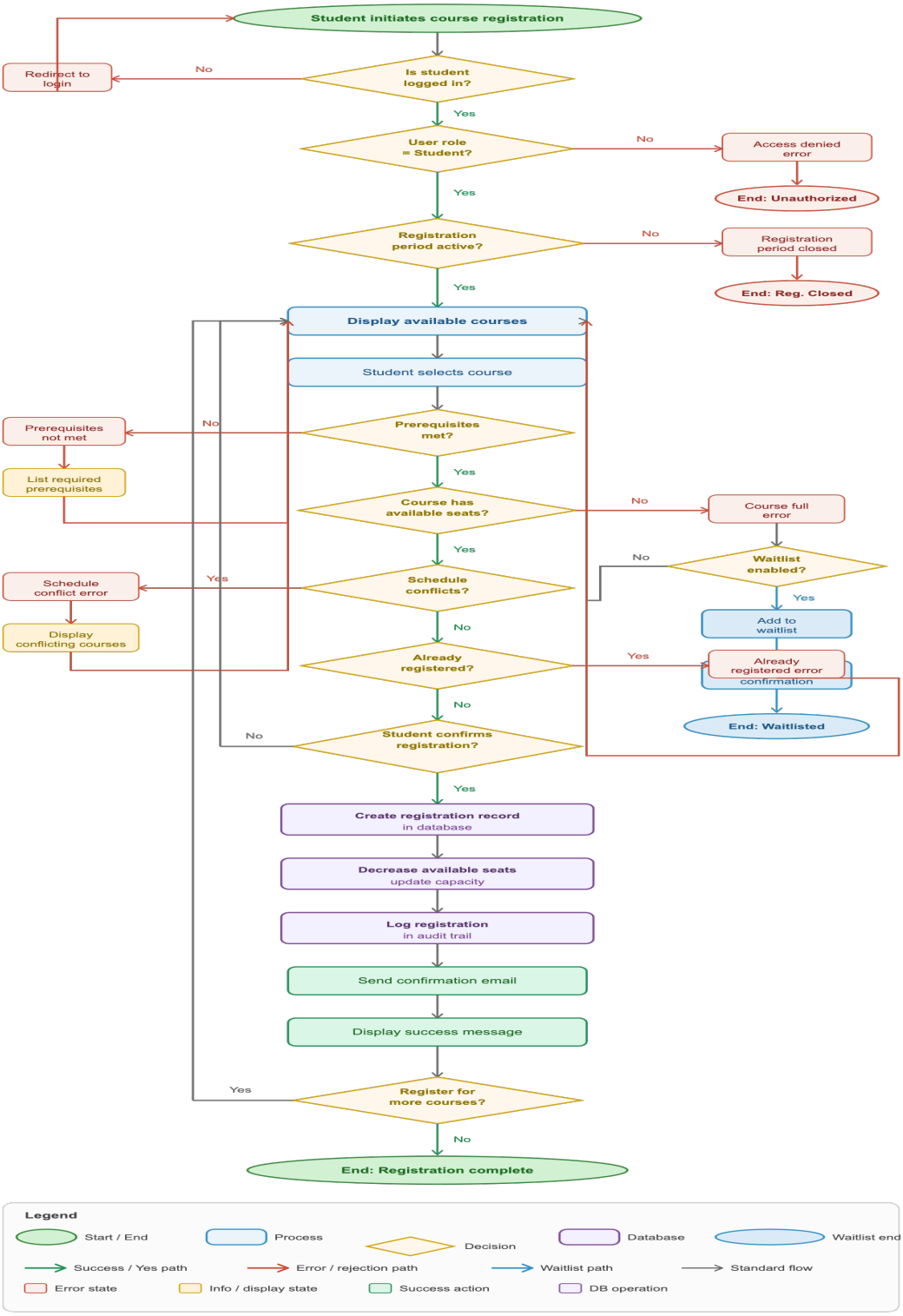
Success Path:

Application submitted → Admin approves → Student number generated → Student account created → Notification sent


Student starts application
Is user logged in?
No
Register account
Navigate to login
Yes
Fill application form
Form data valid?
No
Display validation errors
Yes
Upload required documents
All required docs uploaded?
No
Prompt: upload missing docs
Yes
File types valid?
No
Invalid file format error
Yes
Submit application
Generate reference number
Save to database
Status: Pending
Send confirmation email
Applicant waits for review
Admin reviews application
More info
Send info request
Reject
Update status: Rejected
Log rejection reason
Send rejection email
End: Rejected
Approve
Update status: Approved
Generate student number
Create student record
in database
Update role to "Student"
Log approval
in audit trail
Send approval email with student number
End: Application Approved
Legend
Start / End
Process
Decision
Database
Error
Success / Yes path
Error / No path
Standard flow
Approval actions
Waiting / info states

Flowchart 2: Course Registration Workflow

This flowchart illustrates how students register for courses with thorough validation checks including prerequisites, capacity, and schedule conflicts.



Key Decision Points:

1. **Authentication Check**: Verifies user is logged in
2. **Authorization Check**: Confirms user has Student role
3. **Registration Period Check**: Ensures registration is currently open
4. **Prerequisites Check**: Validates student has completed required prerequisite courses
5. **Capacity Check**: Verifies course has available seats
6. **Schedule Conflict Check**: Ensures no time conflicts with already-registered courses
7. **Duplicate Check**: Prevents registering for same course twice

Error Handling:

- Not logged in → Redirect to login page
- Wrong role → Access denied
- Closed period → Display registration dates
- Prerequisites not met → List required courses
- Course full → Offer waitlist option (if enabled)
- Schedule conflict → Display conflicting courses
- Already registered → Show error message

Success Path: Student authenticated → Checks passed → Registration created → Capacity updated → Confirmation sent

---

Flowchart 3: User Authentication & Authorization Workflow

This flowchart shows how the system authenticates users and grants role-based access to different parts of the application.

Key Decision Points:

1. **Input Validation**: Checks if email and password fields are filled
2. **Format Validation**: Verifies email format is valid
3. **User Existence**: Confirms user exists in database
4. **Account Status**: Ensures account is active (not suspended/deleted)
5. **Password Verification**: Compares entered password with stored hash
6. **Failed Attempts Check**: Locks account after 5 failed login attempts
7. **Role-Based Routing**: Redirects user to appropriate dashboard based on role

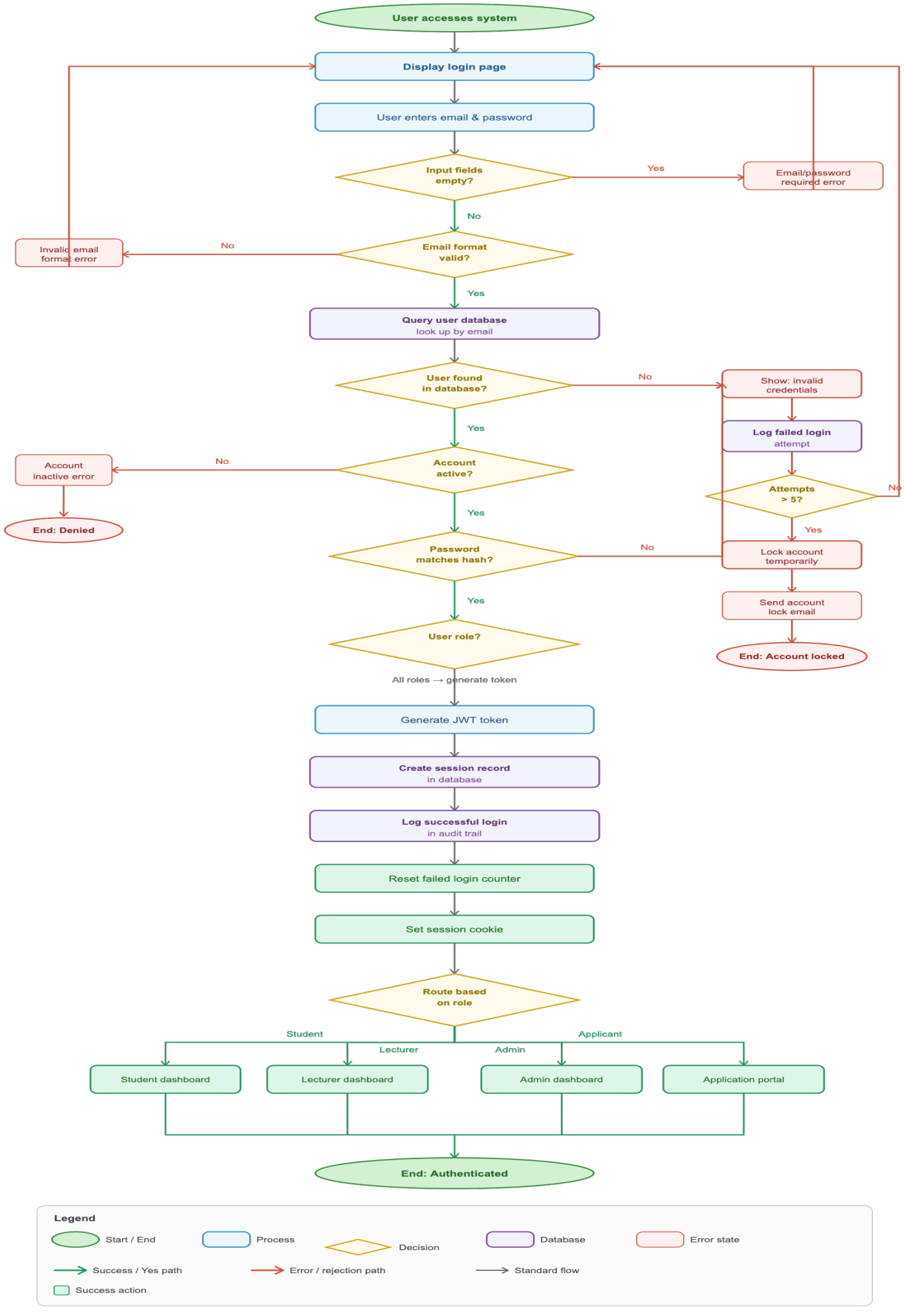
Security Features:

- Password hashing verification (bcrypt)
- Failed login attempt tracking
- Account locking after 5 failed attempts
- Session token generation (JWT)
- Audit trail logging
- Role-based access control

Error Handling:

- Empty fields → Prompt to fill all fields
- Invalid email format → Show format error
- Invalid credentials → Display generic error message (security best practice)
- Account inactive → Show account status message
- Too many failed attempts → Lock account and notify user

Success Path: Credentials validated → Session created → JWT token generated → User redirected to role-specific dashboard


User accesses system
Display login page
User enters email & password
Input fields empty?
Yes
Email/password required error
No
Email format valid?
No
Invalid email format error
Yes
Query user database
look up by email
User found in database?
No
Show: invalid credentials
Log failed login attempt
Attempts > 5?
No
Yes
Lock account temporarily
Send account lock email
End: Account locked
Yes
Account active?
No
Account inactive error
End: Denied
Yes
Password matches hash?
No
Yes
User role?
All roles → generate token
Generate JWT token
Create session record
in database
Log successful login
in audit trail
Reset failed login counter
Set session cookie
Route based on role
Student
Lecturer
Admin
Applicant
Student dashboard
Lecturer dashboard
Admin dashboard
Application portal
End: Authenticated
Legend
Start / End
Process
Decision
Database
Error state
Success / Yes path
Error / rejection path
Standard flow
Success action

Flowchart Summary

Flowchart	Purpose	Decision Points	Complexity	Business Value
Application Submission & Approval	Shows complete application lifecycle	5 major decisions	High	Critical - core admission process
Course Registration	Illustrates registration with validations	8 major decisions	Very High	Critical - core academic function
User Authentication	Details login and role-based access	7 major decisions	High	Essential - security foundation

Key Benefits of These Flowcharts:

1. **Clarity**: Shows exact process flow for developers to build
2. **Error Handling**: Identifies all possible error scenarios and responses
3. **Decision Logic**: Documents all business rules and validation points
4. **Security**: Highlights authentication and authorization checkpoints
5. **User Experience**: Maps out user journey including error recovery paths

These flowcharts complement the existing DFDs (which show data movement) by illustrating **procedural logic and decision-making** within critical system processes.

Entity Relationship Diagram (ERD)

The Entity Relationship Diagram shows the high-level data entities and their relationships.

Primary Entities

The system manages the following core entities:

1. **Users** - All system users with authentication credentials and role information
2. **Students** - Extended information for enrolled students
3. **Applications** - Student application submissions
4. **Application_Documents** - Documents uploaded with applications
5. **Courses** - Course offerings and information
6. **Registrations** - Student course enrollments (junction entity)
7. **Emergency_Contacts** - Student emergency contact information
8. **System_Settings** - System configuration parameters
9. **Audit_Logs** - System audit trail
10. **Notifications** - User notifications

Key Relationships

One-to-One:

- Users ↔ Students (each student record links to one user account)

One-to-Many:

- Users → Applications (users can have multiple applications)
- Students → Emergency_Contacts (students can have multiple emergency contacts)
- Users → Courses (as lecturer - one lecturer can teach multiple courses)
- Applications → Application_Documents (applications can have multiple documents)

Many-to-Many:

- Students ↔ Courses (through Registrations junction table)
    - Students can register for multiple courses
    - Courses can have multiple students enrolled

Entity Descriptions

Users: Central authentication entity storing login credentials, role information, and basic user data for all system users.

Students: Extended profile information for users with Student role, including student number, personal details, program information, and enrollment status.

Applications: Application submissions from applicants, storing application data, status, and tracking information.

Courses: Course catalog information including course code, name, credits, prerequisites, capacity, and assigned lecturer.

Registrations: Junction table linking students to courses, storing enrollment information, registration date, and status.

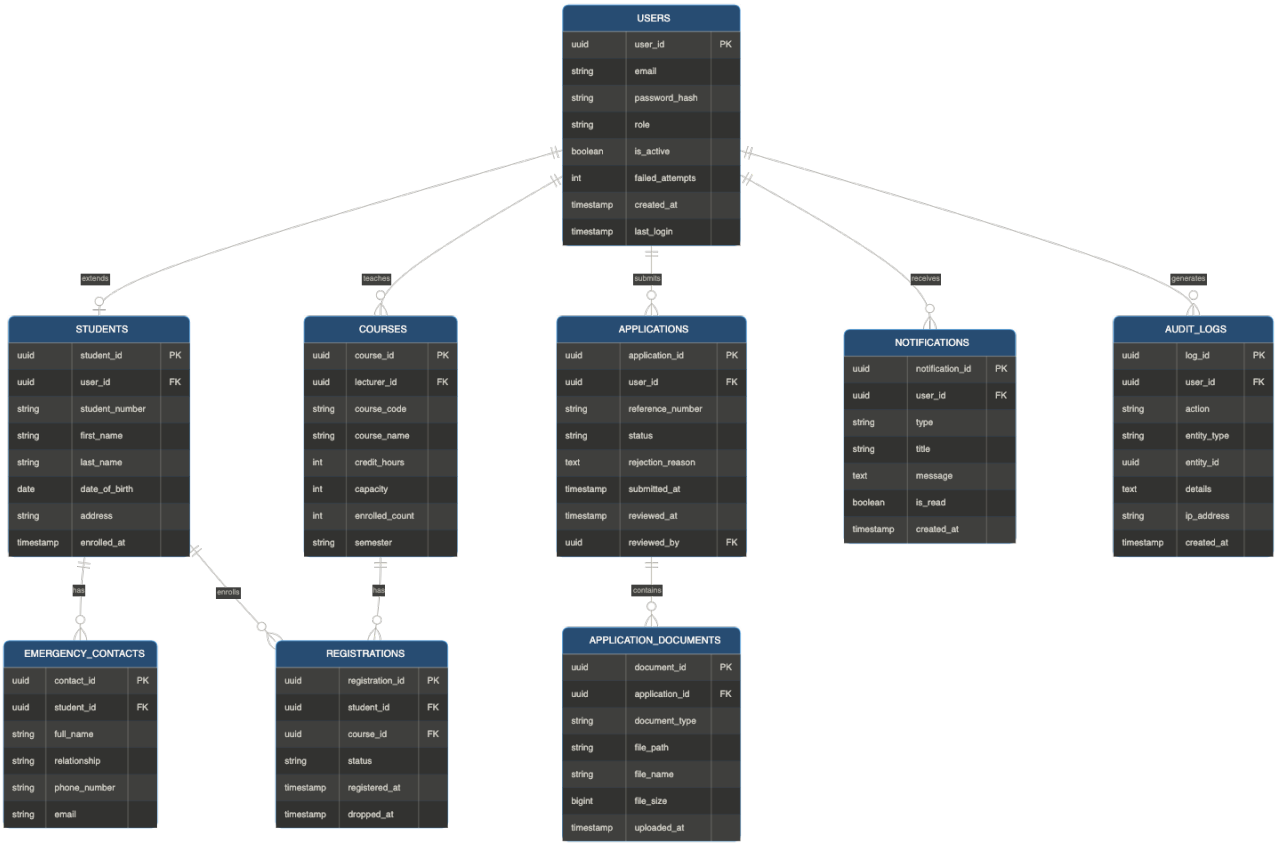
Detailed Entity Relationship Diagram

```
USERS {
    int user_id PK
    string email UK
    string password_hash
    enum role "Student, Lecturer, Administrator"
    string first_name
    string last_name
    date date_of_birth
    string phone_number
    boolean is_active
    datetime created_at
    datetime updated_at
}
```

```
COURSES {
    int course_id PK
    string course_code UK
    string course_name
    text description
    int credits
    int capacity
    string prerequisites
    int lecturer_id FK "References Users"
    string semester
    int year
    boolean is_active
    datetime created_at
    datetime updated_at
}
```

```
STUDENTS {
    int student_id PK
    int user_id FK
    string student_number UK "Auto-generated"
    string id_number
    string address
    string city
    string postal_code
    string program
    string level "Certificate, Diploma, Degree"
    enum status "Active, Inactive, Graduated"
    date enrollment_date
    datetime created_at
    datetime updated_at
  }

  APPLICATIONS {
    int application_id PK
    int user_id FK
    string application_number UK "Auto-generated"
    string program
    string level
    text motivation
    enum status "Pending, Approved, Rejected"
    int reviewed_by FK "References Users"
    datetime reviewed_at
    text review_notes
    datetime created_at
    datetime updated_at
  }

  APPLICATION_DOCUMENTS {
    int document_id PK
    int application_id FK
    enum document_type "ID, Matric, Transcript,
Other"
    string file_name
    string file_path
    int file_size
    string mime_type
    datetime uploaded_at
  }


SYSTEM_SETTINGS {
    int setting_id PK
    string key UK
    string value
    text description
    datetime updated_at
  }
```

```
REGISTRATIONS {
    int registration_id PK
    int student_id FK
    int course_id FK
    enum status "Registered, Dropped, Completed"
    date registration_date
    date drop_date
    string grade
    datetime created_at
    datetime updated_at
  }

  EMERGENCY_CONTACTS {
    int contact_id PK
    int student_id FK
    string contact_name
    string relationship
    string phone_number
    string email
    boolean is_primary
    datetime created_at
    datetime updated_at
  }

  NOTIFICATIONS {
    int notification_id PK
    int user_id FK
    string title
    text message
    enum type "Info, Warning, Success, Error"
    boolean is_read
    datetime created_at
    datetime read_at
  }

  AUDIT_LOGS {
    int log_id PK
    int user_id FK
    string action
    string entity_type
    int entity_id
    json old_values
    json new_values
    string ip_address
    datetime created_at
  }
```



Entity Attribute Details

USERS Table

Attribute	Type	Constraints	Description
user_id	INTEGER	PK, AUTO_INCREMENT	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email (login username)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role	ENUM	NOT NULL	User role (Student, Lecturer, Administrator)
first_name	VARCHAR(100)	NOT NULL	User first name
last_name	VARCHAR(100)	NOT NULL	User last name
date_of_birth	DATE	NOT NULL	User date of birth
phone_number	VARCHAR(20)	-	Contact phone number
is_active	BOOLEAN	DEFAULT TRUE	Account active status
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

STUDENTS Table

Attribute	Type	Constraints	Description
student_id	INTEGER	PK, AUTO_INCREMENT	Unique student identifier
user_id	INTEGER	FK (USERS), UNIQUE	Reference to Users table
student_number	VARCHAR(20)	UNIQUE, NOT NULL	Auto-generated student number
id_number	VARCHAR(20)	UNIQUE, NOT NULL	National ID number
address	VARCHAR(255)	-	Residential address
city	VARCHAR(100)	-	City
postal_code	VARCHAR(10)	-	Postal code
program	VARCHAR(100)	NOT NULL	Study program
level	ENUM	NOT NULL	Certificate, Diploma, Degree
status	ENUM	DEFAULT 'Active'	Active, Inactive, Graduated
enrollment_date	DATE	NOT NULL	Date of enrollment
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

APPLICATIONS Table

Attribute	Type	Constraints	Description
application_id	INTEGER	PK, AUTO_INCREMENT	Unique application identifier
user_id	INTEGER	FK (USERS)	Reference to applicant
application_number	VARCHAR(20)	UNIQUE, NOT NULL	Auto-generated reference number
program	VARCHAR(100)	NOT NULL	Applied program
level	ENUM	NOT NULL	Certificate, Diploma, Degree
motivation	TEXT	-	Motivation statement
status	ENUM	DEFAULT 'Pending'	Pending, Approved, Rejected
reviewed_by	INTEGER	FK (USERS)	Administrator who reviewed
reviewed_at	TIMESTAMP	-	Review timestamp
review_notes	TEXT	-	Admin review notes
created_at	TIMESTAMP	DEFAULT NOW()	Application submission timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

COURSES Table

Attribute	Type	Constraints	Description
course_id	INTEGER	PK, AUTO_INCREMENT	Unique course identifier
course_code	VARCHAR(20)	UNIQUE, NOT NULL	Course code (e.g., CS101)
course_name	VARCHAR(200)	NOT NULL	Course name
description	TEXT	-	Course description

credits	INTEGER	NOT NULL	Credit hours
capacity	INTEGER	NOT NULL	Maximum enrollment
prerequisites	VARCHAR(255)	-	Prerequisite courses
lecturer_id	INTEGER	FK (USERS)	Assigned lecturer
semester	VARCHAR(20)	NOT NULL	Semester (1, 2, Summer)
year	INTEGER	NOT NULL	Academic year
is_active	BOOLEAN	DEFAULT TRUE	Course active status
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

REGISTRATIONS Table (Junction Table)

Attribute	**Type**	**Constraints**	**Description**
registration_id	INTEGER	PK, AUTO_INCREMENT	Unique registration identifier
student_id	INTEGER	FK (STUDENTS)	Reference to student
course_id	INTEGER	FK (COURSES)	Reference to course
status	ENUM	DEFAULT 'Registered'	Registered, Dropped, Completed
registration_date	DATE	NOT NULL	Date of registration
drop_date	DATE	-	Date course was dropped (if applicable)
grade	VARCHAR(5)	-	Final grade (populated later)
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

Note: The combination of student_id and course_id should have a unique constraint to prevent duplicate registrations.

Cardinality Summary

Relationship	**Type**	**Description**
Users ↔ Students	1:1	One user account can have one student profile
Users → Applications	1:N	One user can submit multiple applications
Users → Courses	1:N	One lecturer can teach multiple courses
Students → Emergency Contacts	1:N	One student can have multiple emergency contacts
Applications → Documents	1:N	One application can have multiple documents
Students ↔ Courses	M:N	Students enroll in multiple courses; courses have multiple students (through Registrations)

Database Normalization

The database design follows

Third Normal Form (3NF):

- **1NF**: All attributes contain atomic values

- **2NF**: No partial dependencies (all non-key attributes depend on the entire primary key)

- **3NF**: No transitive dependencies (non-key attributes depend only on the primary key)

Detailed schema with indexes, constraints, and performance optimizations will be finalized in Phase 4 - System Design.

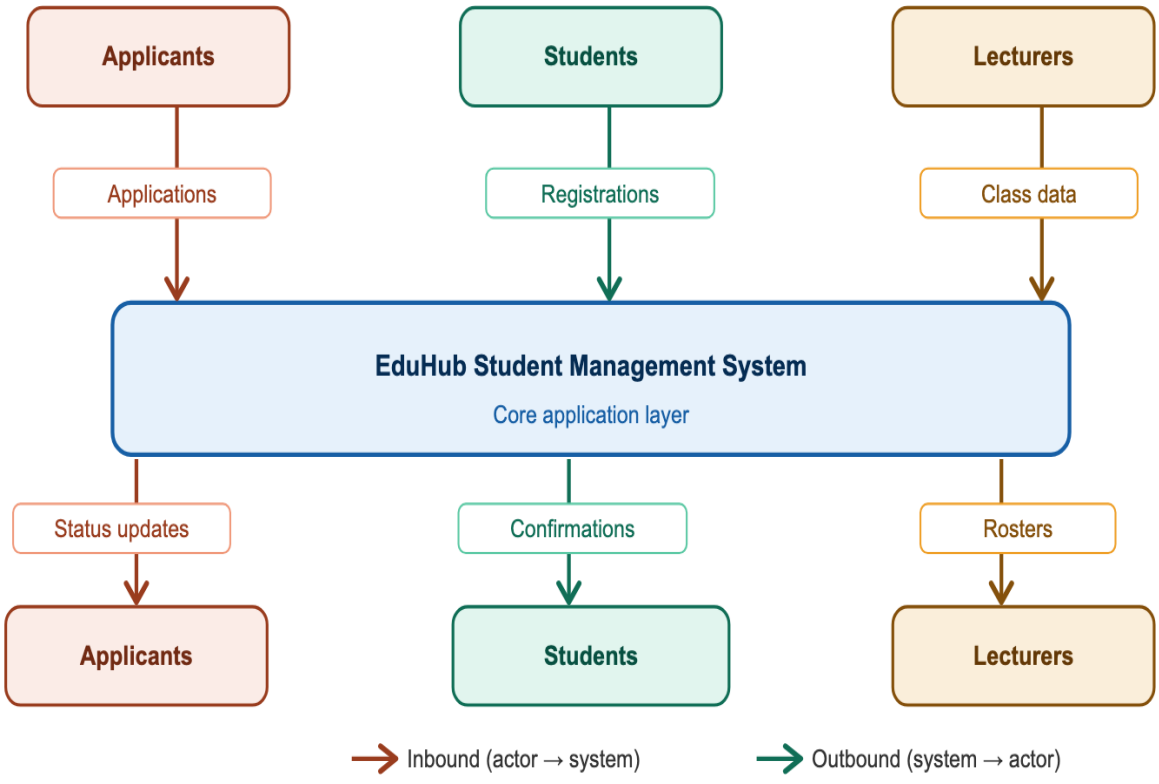
Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the system, showing processes, data stores, and information flows.

DFD Level 0 (Context Diagram)

The context diagram shows the system boundary and external entities:

External Entities:



Input Data Flows:

- Applicants: Application data, documents
- Students: Profile updates, course selections
- Lecturers: Course information, announcements
- Administrators: Approval decisions, system configuration

Output Data Flows:

- To Applicants: Application status, notifications
- To Students: Registration confirmations, course information
- To Lecturers: Class rosters, student contact details
- To Administrators: Reports, analytics, system data

Major System Processes

The EduHub system consists of the following high-level processes:

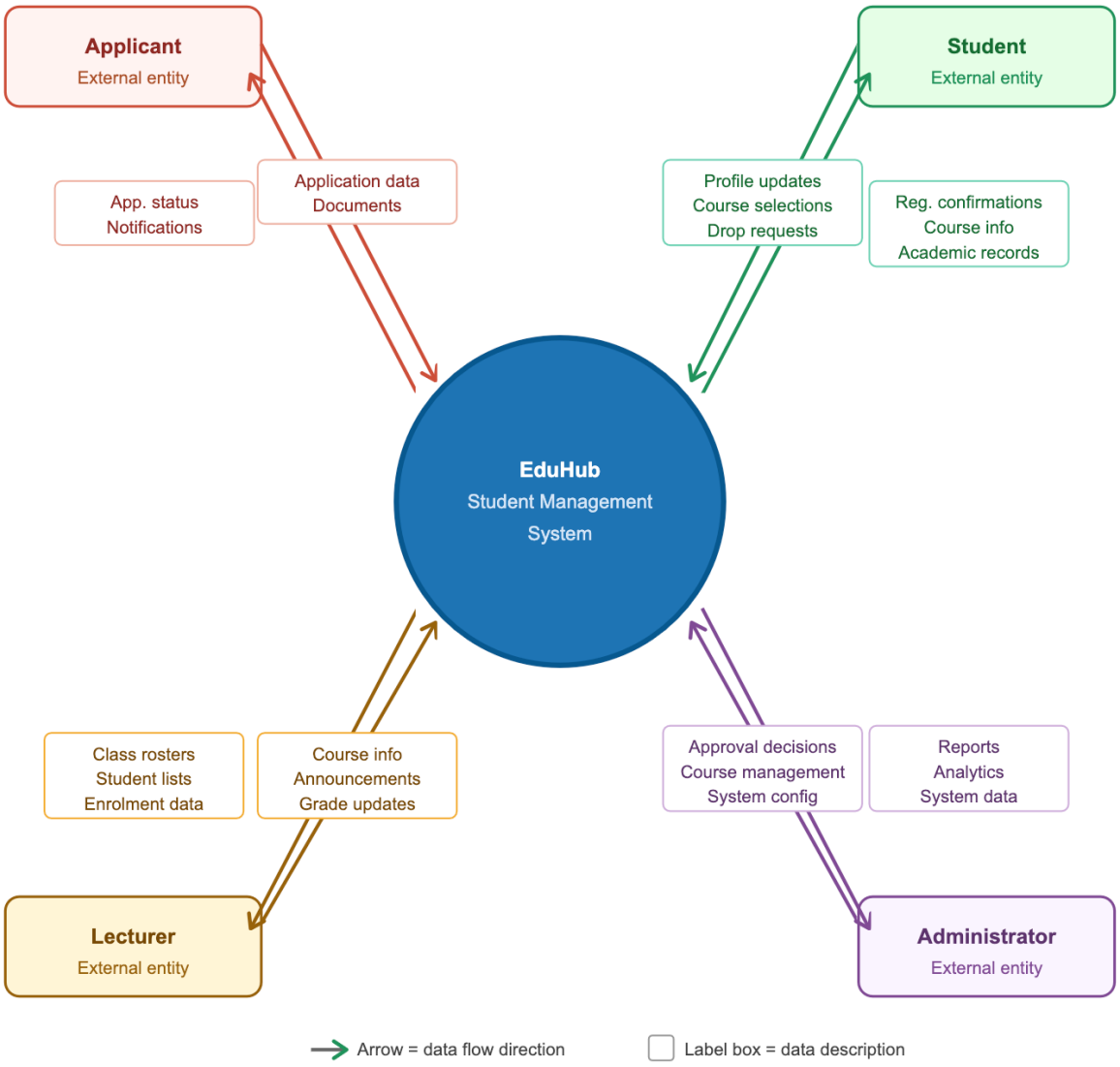
1. **User Authentication** - Validates credentials, manages sessions
2. **Application Management** - Handles application submission and tracking
3. **Application Approval** - Processes admin reviews and decisions
4. **Profile Management** - Manages student information updates
5. **Course Registration** - Handles course enrollments and drops
6. **Course Management** - Manages course creation and assignments
7. **Notification Management** - Sends alerts and notifications
8. **Reporting** - Generates system reports and analytics

Data Stores

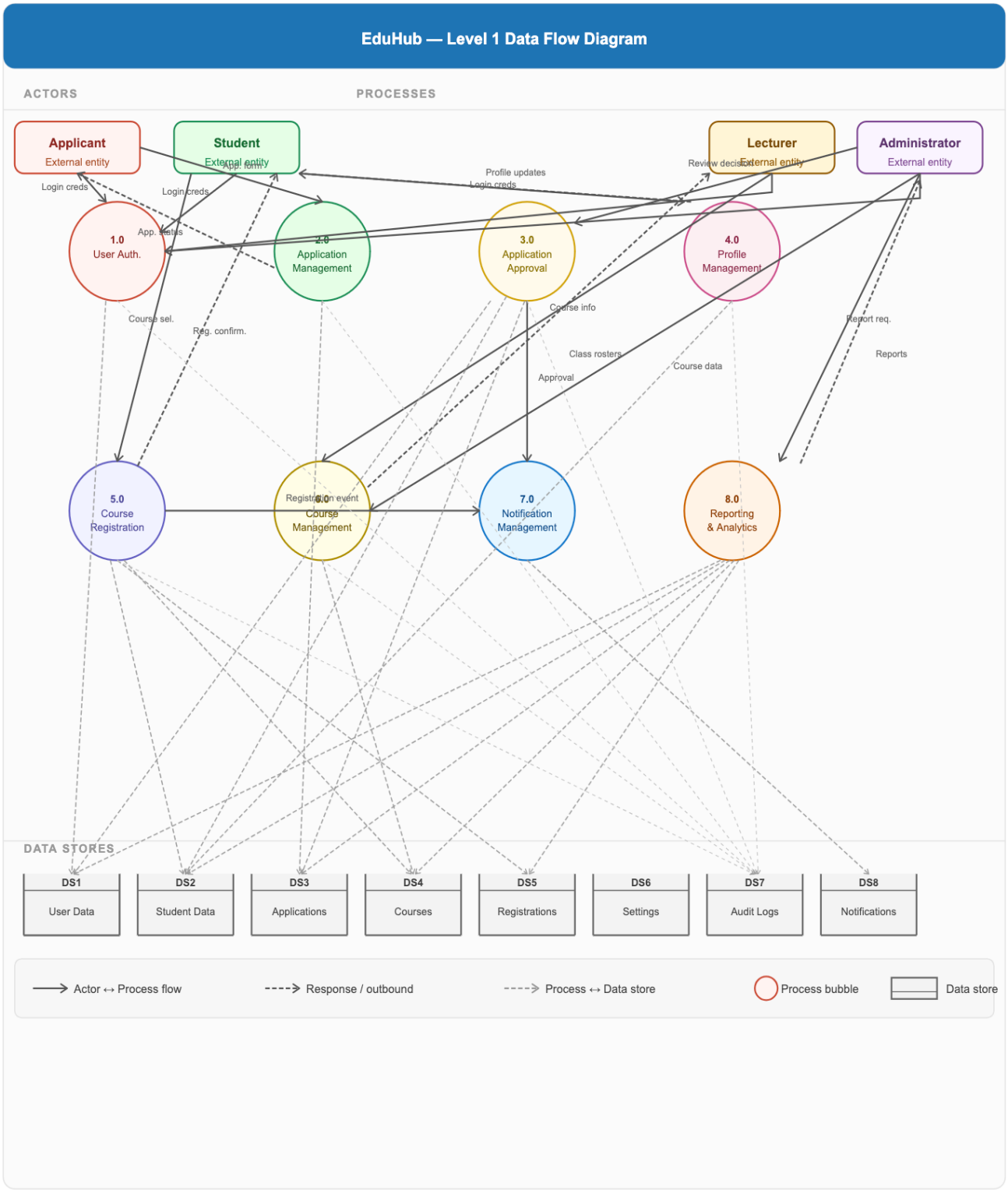
The system uses the following conceptual data stores:

- **DS1: User Data** - Authentication and user account information
- **DS2: Student Data** - Student profiles and academic records
- **DS3: Application Data** - Application submissions and documents
- **DS4: Course Data** - Course catalog and offerings
- **DS5: Registration Data** - Student-course enrollments
- **DS6: System Configuration** - Settings and parameters
- **DS7: Audit Trail** - System activity logs
- **DS8: Notifications** - User alerts and messages

DFD Level 0 - Context Diagram (Visual)



DFD Level 1 - Major System Processes



Process Descriptions

Process 1.0: User Authentication

Input: Login credentials (email, password)
Processing:

- Validates credentials against user database
- Checks user role and permissions
- Creates session token
- Logs authentication attempt

Output: Authentication token, user role information
Data Stores Used: DS1 (User Data), DS7 (Audit Logs)

Process 2.0: Application Management

Input: Application form data, uploaded documents
Processing:

- Validates application data
- Generates application reference number
- Stores application and documents
- Updates application status

Output: Application confirmation, reference number
Data Stores Used: DS3 (Applications), DS7 (Audit Logs)

Process 3.0: Application Approval

Input: Review decision from administrator
Processing:

- Validates admin permissions
- Updates application status
- If approved: generates student number, creates student record, updates user role
- If rejected: logs reason
- Triggers notification

Output: Approval/rejection notification
Data Stores Used: DS1 (User Data), DS2 (Student Data), DS3 (Applications), DS7 (Audit Logs)

Process 4.0: Profile Management

Input: Profile update requests, search queries
Processing:

- Validates user permissions
- Updates student profile information
- Manages emergency contacts

Output: Updated profile data, search results
Data Stores Used: DS2 (Student Data), DS7 (Audit Logs)

Process 5.0: Course Registration

Input: Course selection, drop requests
Processing:

- Checks prerequisites
- Verifies course capacity
- Checks schedule conflicts
- Creates/updates registration record
- Triggers confirmation notification

Output: Registration confirmation, enrollment status
Data Stores Used: DS2 (Student Data), DS4 (Courses), DS5 (Registrations), DS7 (Audit Logs)

Process 6.0: Course Management

Input: Course creation/update requests, lecturer assignments
Processing:
- Validates admin permissions
- Creates/updates course records
- Assigns lecturers to courses
- Manages course capacity and schedule

Output: Course information, class rosters
Data Stores Used: DS4 (Courses), DS7 (Audit Logs)

Process 7.0: Notification Management

Input: Event triggers (approval, registration, etc.)
Processing:
- Creates notification records
- Sends email notifications
- Manages notification read/unread status

Output: Email notifications, in-app notifications
Data Stores Used: DS8 (Notifications)

Process 8.0: Reporting & Analytics

Input: Report requests, date ranges, filters
Processing:
- Queries relevant data stores
- Aggregates data
- Generates reports (applications, enrollments, usage)
- Exports to CSV/PDF

Output: Statistical reports, analytics dashboards
Data Stores Used: DS1-DS5 (All data stores for reporting)

Data Flow Examples

Application Submission Flow

Applicant fills form → Validates data → Stores in database →
Sends confirmation email → Updates application status

Course Registration Flow

Student selects course → Checks prerequisites → Checks capacity →
Checks conflicts → Creates registration → Sends confirmation

Approval Workflow Flow

Admin reviews application → Makes decision → Generates student number →
Creates student record → Updates user role → Sends notification

Data Model Summary

The EduHub system's conceptual data model includes:

- **10 primary entities** managing users, students, applications, courses, and system data
- **Multiple relationship types** including one-to-one, one-to-many, and many-to-many
- **8 major processes** handling authentication, applications, profiles, courses, and reporting
- **8 data stores** persisting system information

Conclusion

The planning phase has identified the need for the EduHub system, evaluated its feasibility, planned project activities, defined requirements, and prepared scheduling structures for development. These planning activities provide the foundation for the next phases of the project, including system analysis and system design.

This conceptual foundation will guide the development of detailed database schemas, API specifications, and system architecture in subsequent project phases.